

Kafka Fundamentals

Agenda – Day 1

- Introduction to messaging system
- Kafka Basics
- Setup
- CLI
- Kafka Java
- Configurations
- Schema Registry
- ACL

Scenario

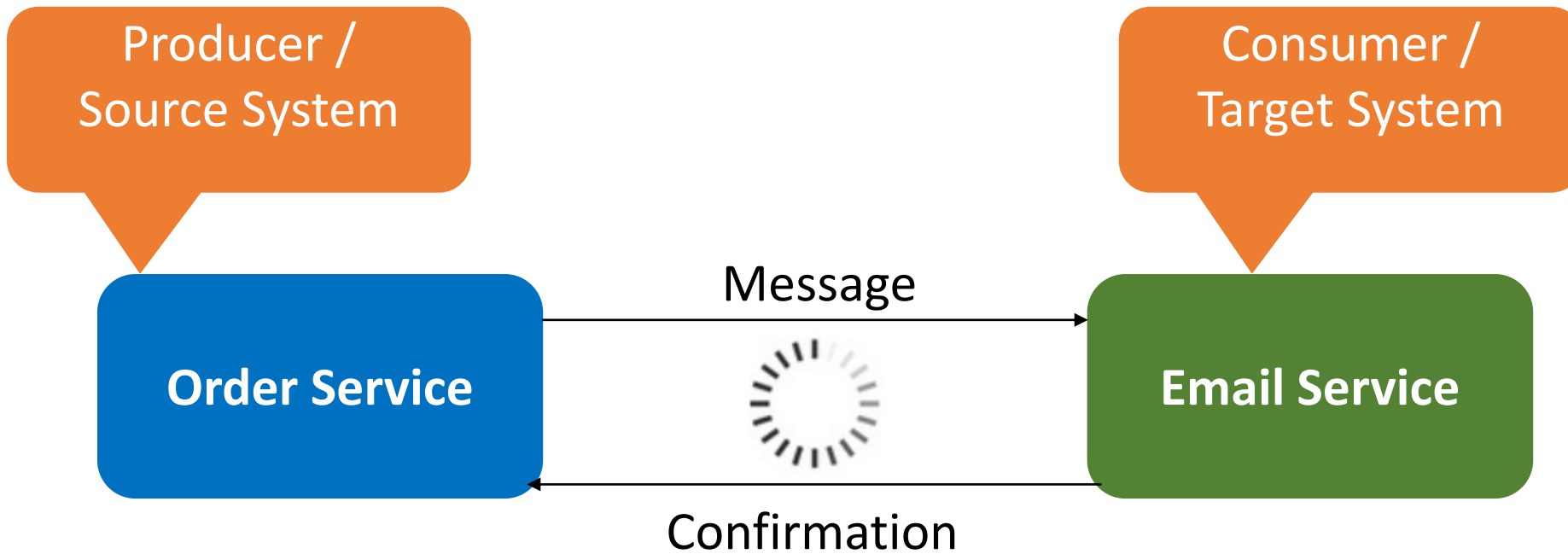
- In an E-commerce application requirement is to send an email once user makes an order

Traditional synchronous way

Flow will be like this –

- User submits an Order
- Order gets created
- Send order confirmation email to user

Traditional synchronous way



Problems with traditional approach

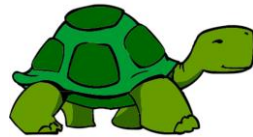
Problems with traditional approach

- Both producer and consumer must be available for the duration of the request
- What happens if email service is down? **Entire flow may fail** though email service may not be the key component for the order creation flow.
- Loss of business



Problems with traditional approach

- **Performance issue** as source system need to wait for the email to be send before showing confirmation to user
- In this case Order Service need to wait for the Email Service
- Email service can be a bottle neck (A slow consumer can slow down entire flow)

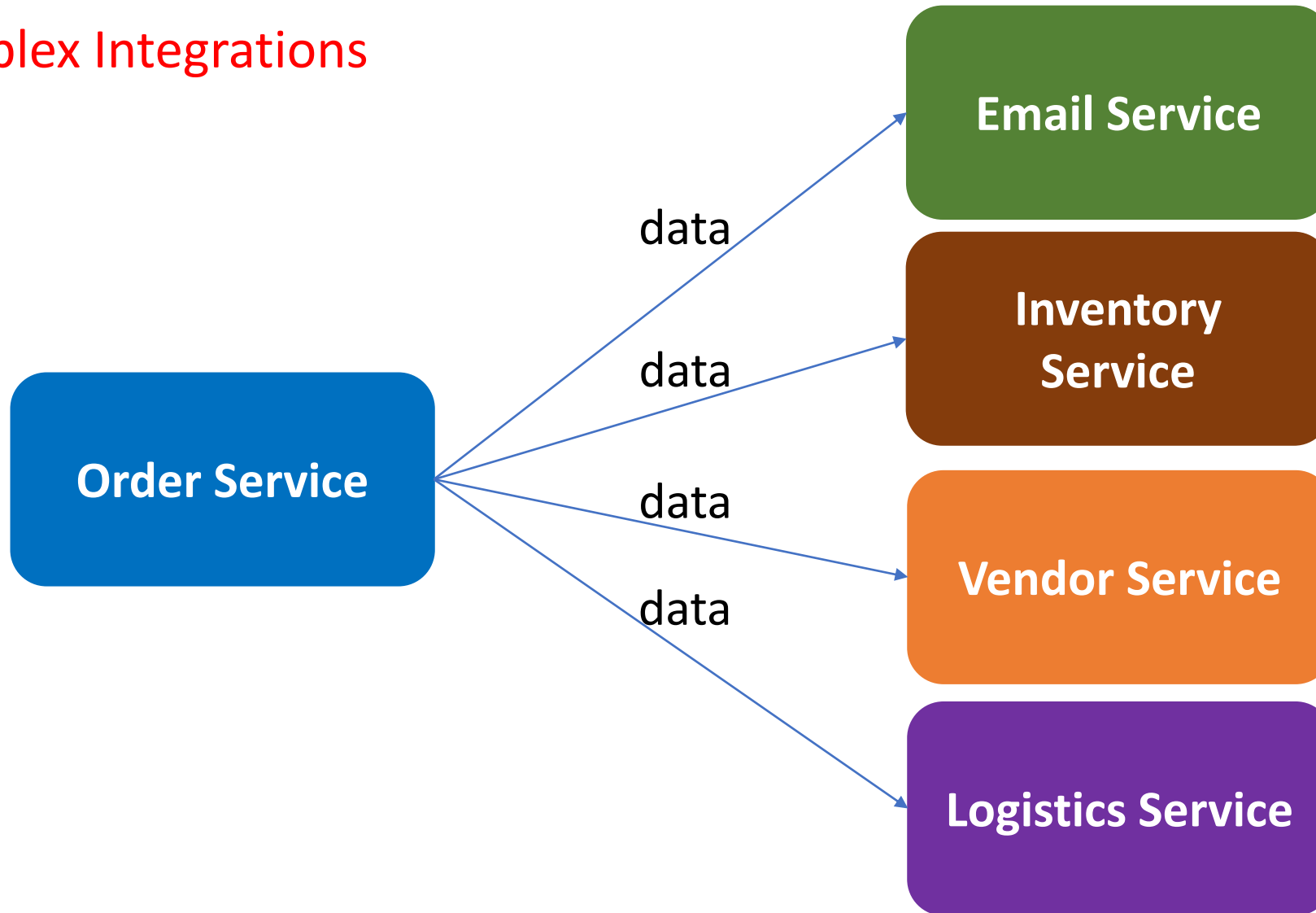


LOADING

Problems with traditional approach

- **Complex Integrations** – What happens if we have multiple Consumers? (Ex. Once order creates, I want to send confirmation email and alert Inventory service, Vendor service, Logistics service etc.)
- Source application need to send messages to all required applications

Complex Integrations



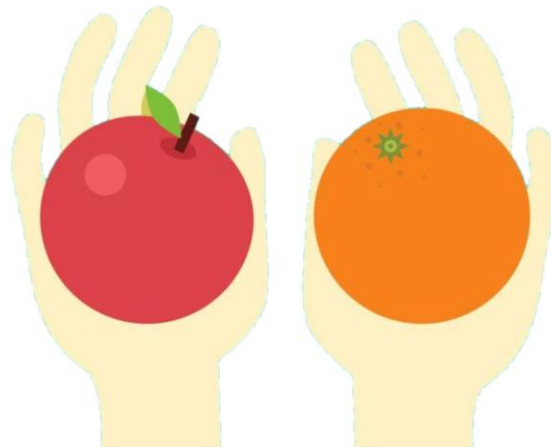
Problems with traditional approach

- **Tight coupling** between producer and consumer –
- What happens if Consumer code deployed with some errors – though Producer has nothing to do with it, it fails



Problems with traditional approach

- **Limited technology choice** – Because of tight coupling technology options to choose are limited



Can we fix it using asynchronous messaging?



Asynchronous Messaging

- Yes - By simply passing the message to Email service and continue with our flow
- But - this is partial solution only
- **What happens if email delivery fails?** We don't have a way to know the status (or) resend the message
- Still problems like – tight coupling, limited technology choice, complexity exists + an additional problem – system not reliable

Is there any other way?



*Yes. We can use **Messaging System***



Messaging system

What is a messaging system?

- A **Messaging System** is responsible for transferring data from one application to another, so the applications can focus on data, but not worry about how to share it
- **Ex.** Order Service in an E-commerce application need to send some data to downstream systems like Email service, Inventory service, Logistics service, Sales analytic service etc.

What is a messaging system?

- We can achieve Asynchronous application-to-application communication using a messaging system

Parts of a Messaging System

- **Sender (Producer)** – Application which produces the message
- **Receiver (Consumer)** – Application which consumes the message
- **Queue** – Message store

Messaging System - Types

- **Producer-Consumer Model** (or point-to-point messaging model)
- **Publish - Subscriber Model** (or broadcast messaging model)

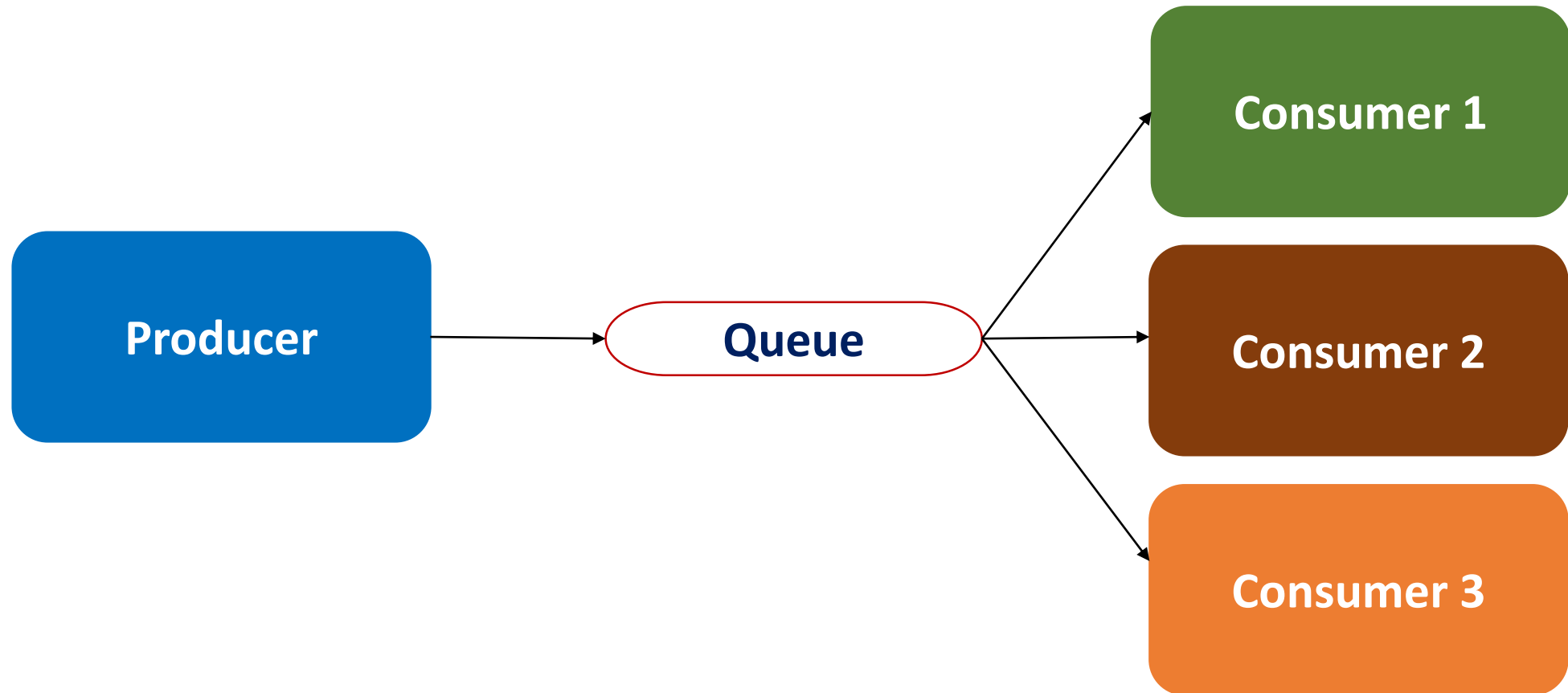
Point-to-point messaging model

- Here message will be received by only one Consumer



Publish - Subscriber messaging model

- Here message can be received by more than one Consumer



Messaging System - Advantages

- Enables asynchronous communication between applications
- Decouples the applications - Any changes made to one application doesn't affect other application as long as communication contract is not breached (We can think of contract as an agreement of the message format)
- Pro microservice - Helps in developing micro services
- Scalable (How? – through Topics, Partitions, Consumer Groups etc.)

Messaging System - Advantages

- **Reliable** (Producers need not wait for Consumer. Simply push messages to Queue. Consumer once available starts consuming)
- **Fault tolerant** – Message Queue persists messages until it gets consumed. So even if some components are down , system becomes fault tolerant as no data loss
- **Multiple technology choices** (We can use different technologies for different applications - Ex. Producer can be in .Net, Consumer can be in Java. It doesn't matter)

Messaging System - Examples

ActiveMQ

RabbitMQ

Kafka

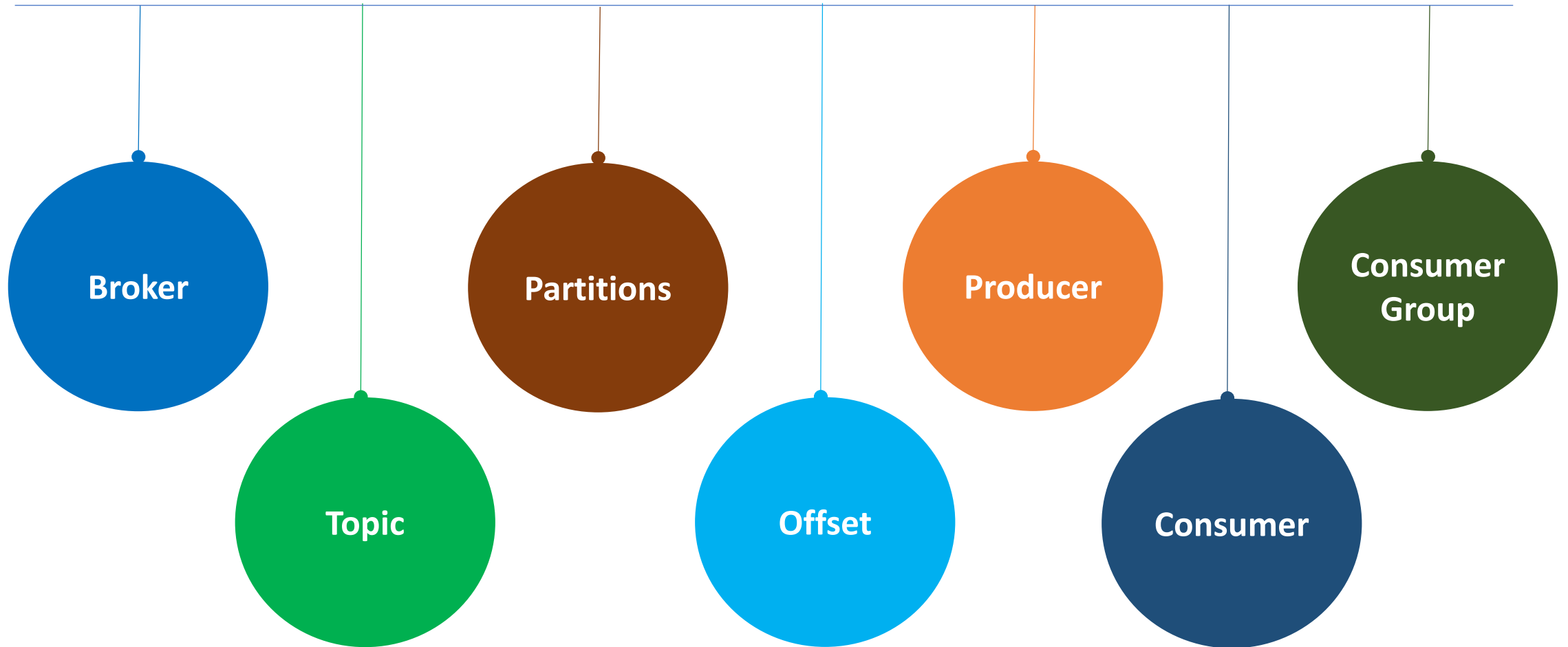
Kafka Fundamentals



What is Kafka?

*Apache Kafka is a **distributed messaging system***

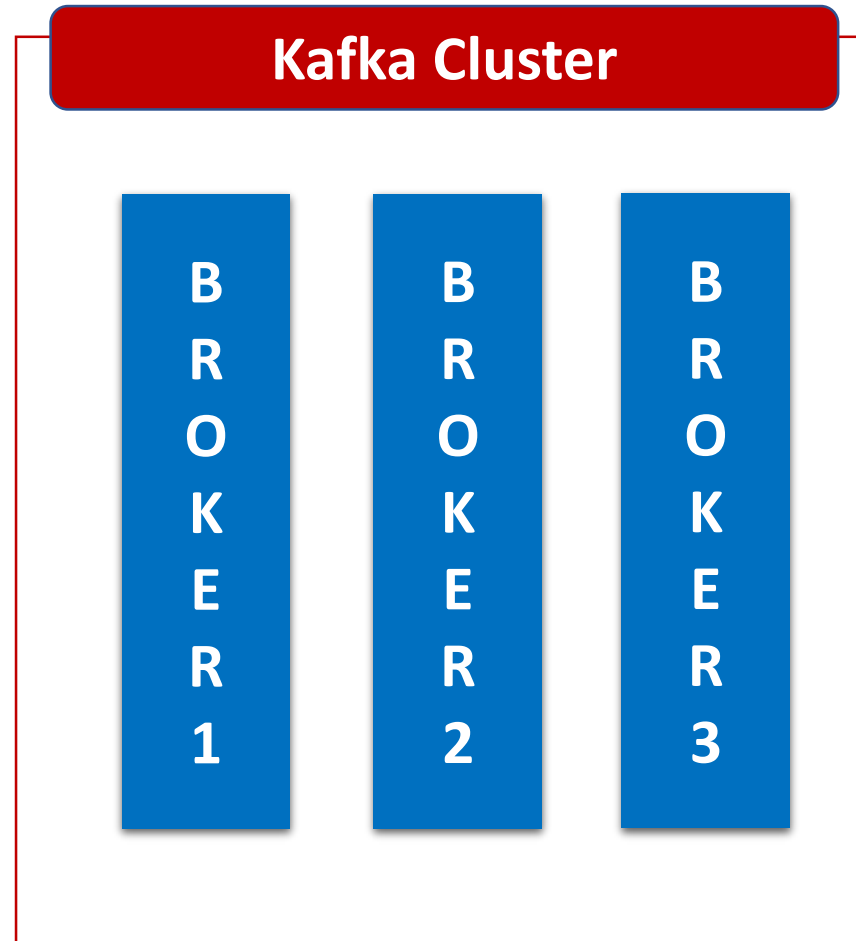
Kafka Terminology



Kafka Cluster & Broker

- **Kafka cluster** is nothing but a group of servers
- **Broker** - Each server in Kafka cluster is called as a broker
- They are responsible for maintaining (storing) the published messages
- Each broker may have zero or more partitions per topic
- In general, number of Brokers in a Kafka cluster is 3
- But depending on requirement, a Kafka cluster can have 100s of Brokers

Example – Kafka Cluster with 3 Brokers



Kafka Cluster & Broker

- Brokers store the published messages
- Each broker may have zero or more partitions per topic
- In general, number of Brokers in a Kafka cluster is 3
- But depending on requirement, a Kafka cluster can have 100s of Brokers

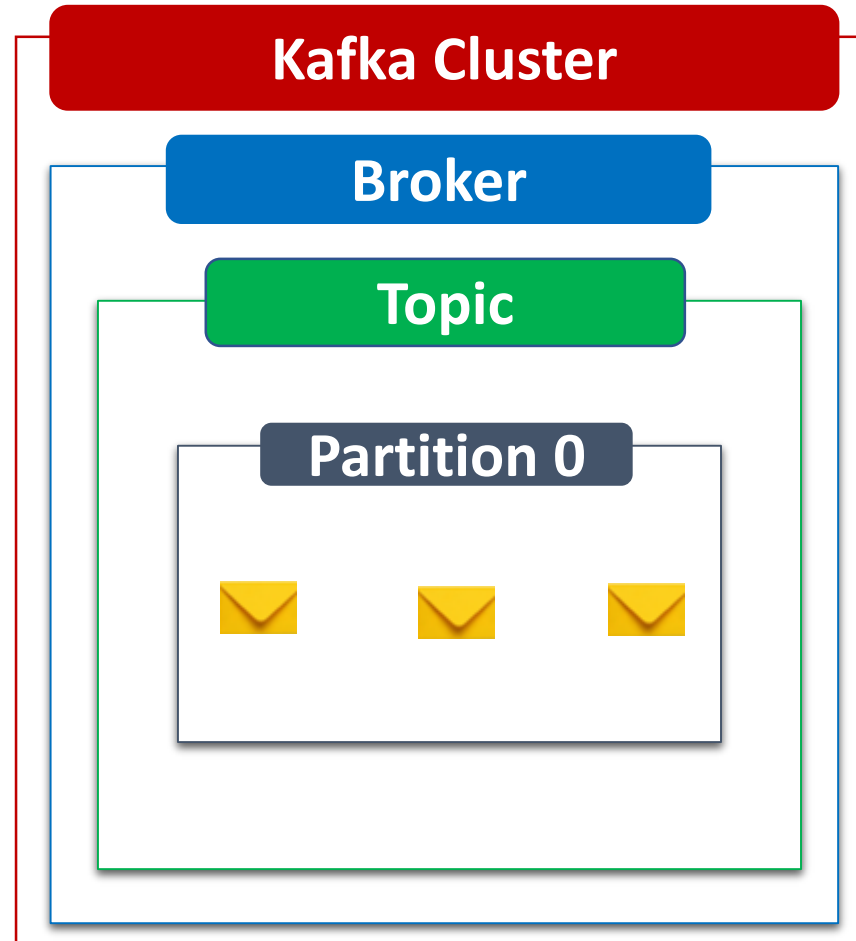
Topics

- *A stream of messages belonging to a particular category is called a **Topic***
- Ex. Order Information in our example is a Topic
- Each Topic is represented by a name
- Data is stored in topics
- We can try to visualize Topic as a Table in a database.
 - As all the records belonging to the table gets stored in that Table, all the messages belonging to a Topic stored in that Topic

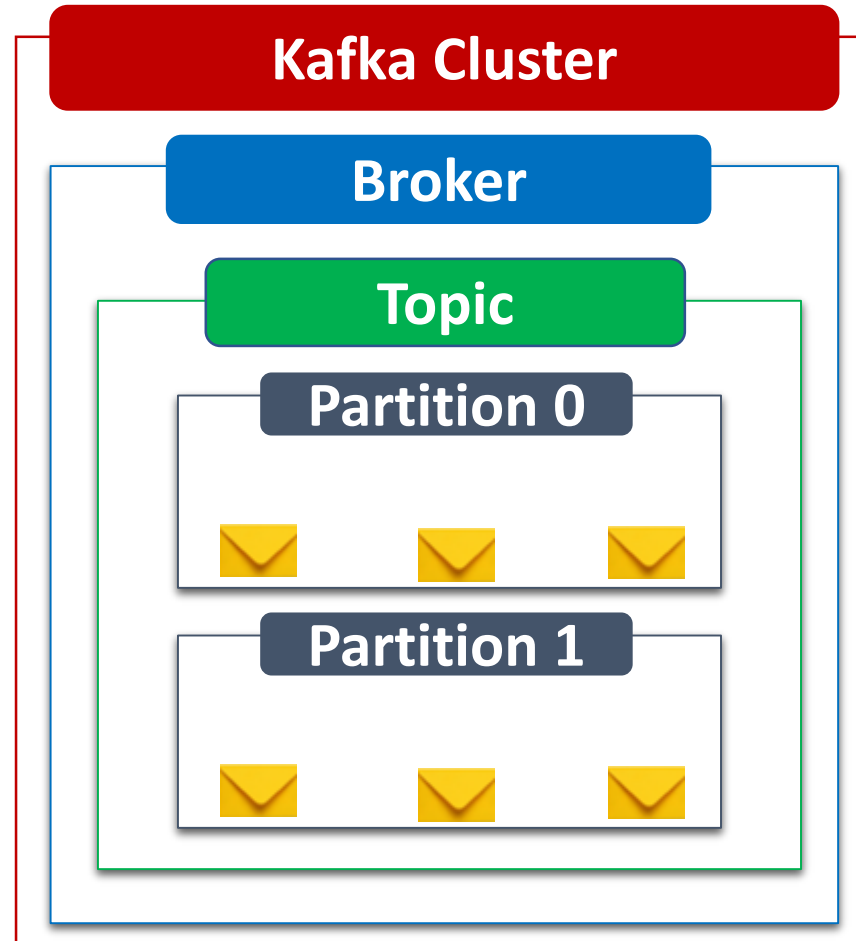
Partition

- **Partition** - Topics are split into partitions
- In Kafka each Topic keeps minimum one partition
- Partitions related to a Topic are stored in Brokers
- Each broker may have zero or more partitions per topic

Example – Kafka Cluster with 1 Broker, 1 Topic, 1 Partition



Example – Kafka Cluster with 1 Broker, 1 Topic, 2 Partitions



Partition

- Each partition contains messages in an immutable, ordered sequence
- **Immutable** meaning once the message is posted to Kafka Topic its content can't be changed
- **Ordered** means – Kafka guarantees ordered delivery of messages within a partition
- Unless a partition key is specified while sending message, message gets assigned to a random partition

Offset

- Each message within a partition has a unique sequence id called offset
- Kafka guarantees ordered delivery of messages within a partition

Offset

Topic

Partition 0

0 	1 	2 	3 
---	---	---	---

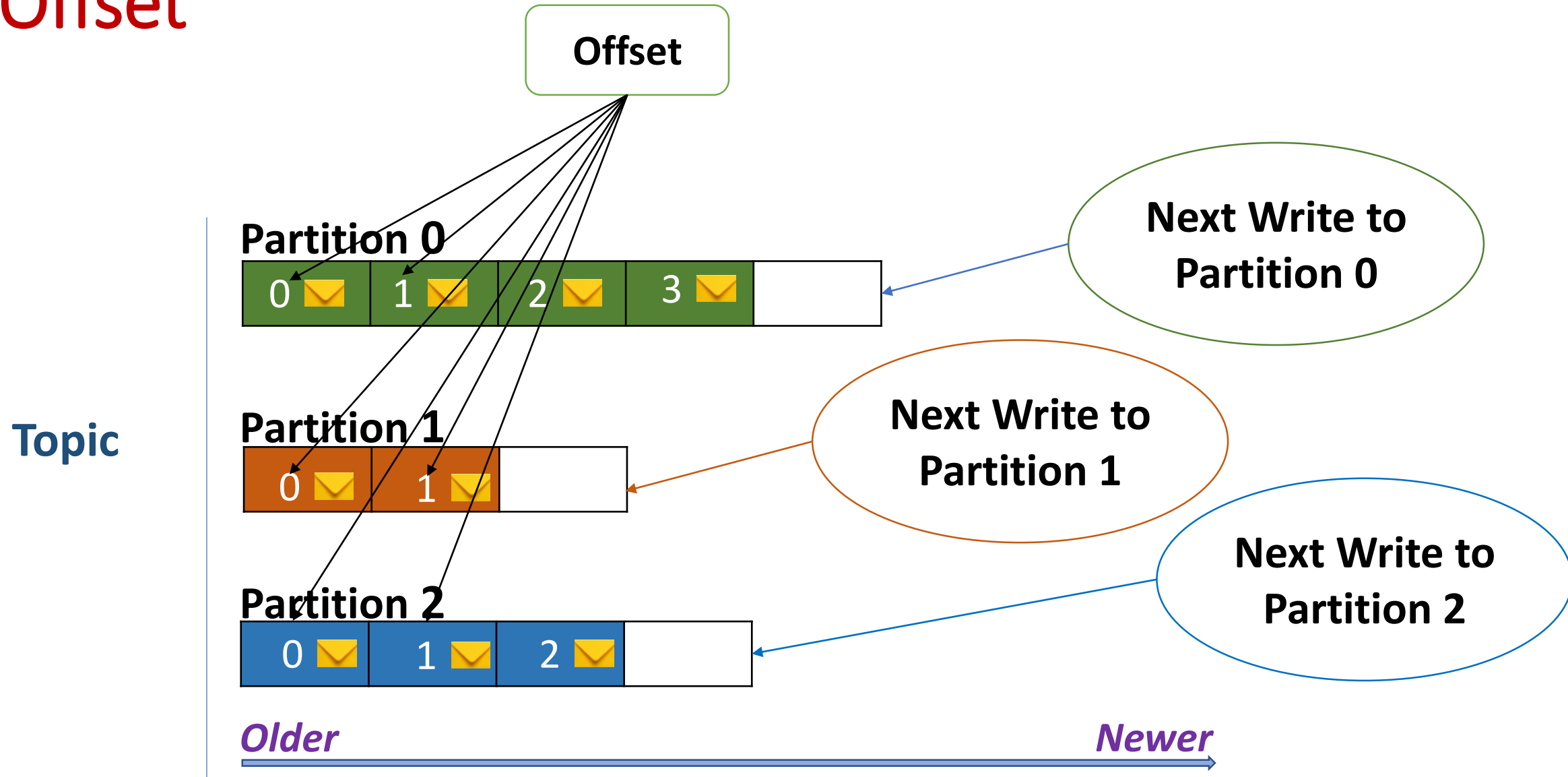
Partition 1

0 	1 
---	---

Partition 2

0 	1 	2 
---	---	---

Offset



**Next read
from
Partition 0**

**Next read
from
Partition 1**

**Next read
from
Partition 2**

Partition 0



Partition 1



Partition 2



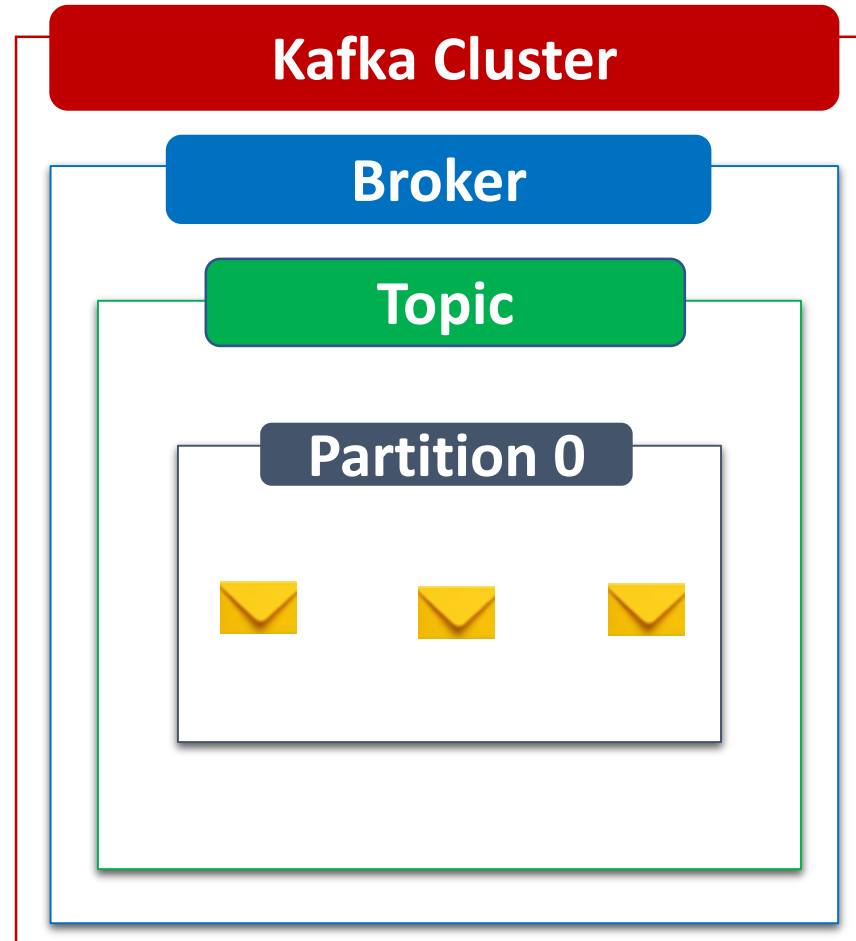
Brokers and topics

- Let us try to check different combinations of Broker-Topic-Partitions for better understanding

Brokers and topics

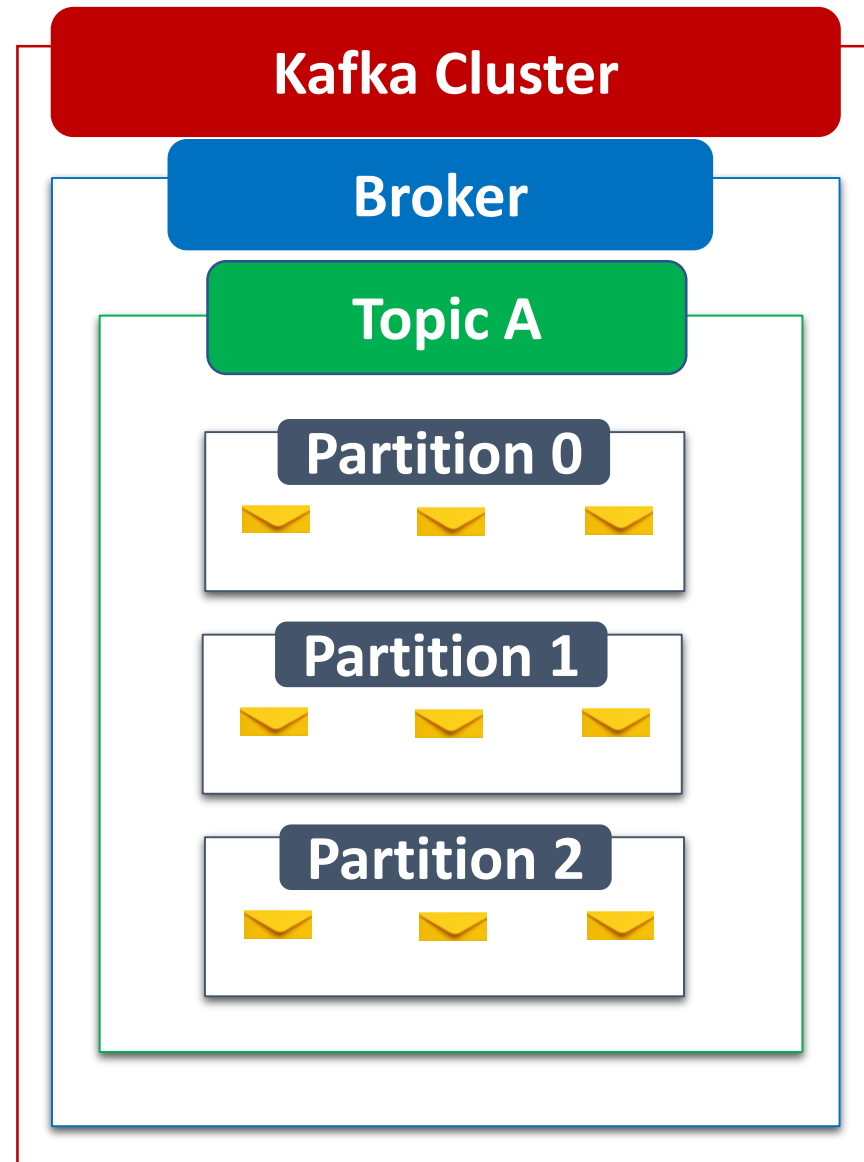
- Kafka Cluster with 1 Broker
- Topic A with 1 Partition

Kafka Cluster with 1 Broker, 1 Topic, 1 Partition



Brokers and topics

- Kafka Cluster with 1 Broker
- Topic A with 3 Partitions



Brokers and topics

- Kafka Cluster with 2 Brokers
- Topic A with 3 Partitions

Kafka Cluster

Broker 0

Topic A

Partition 0



Partition 2



Broker 1

Topic A

Partition 1



Brokers and topics

- Kafka Cluster with 3 Brokers
- Topic A with 3 Partitions

Kafka Cluster

Broker 0

Topic A

Partition 0



Broker 1

Topic A

Partition 1



Broker 2

Topic A

Partition 2



Brokers and topics

- Kafka Cluster with 4 Brokers
- Topic A with 3 Partitions

Kafka Cluster

Broker 0

Topic A

Partition 0



Broker 1

Topic A

Partition 1



Broker 2

Topic A

Partition 2



Broker 4

Brokers and topics

- Kafka Cluster with 4 Brokers
- Multiple Topics - Topic A with 3 Partitions, Topic B with 4 partitions etc.

Kafka Cluster

Broker 0

Topic A

Partition 0



Topic B

Partition 0



Broker 1

Topic A

Partition 1



Topic B

Partition 1



Broker 2

Topic A

Partition 2



Topic B

Partition 2



Broker 3

Topic B

Partition 3



Producer

- Producers are the publisher of messages to one or more topics
- They send messages to Kafka brokers
- Messages belonging to a Topic will be placed in partitions of that topic

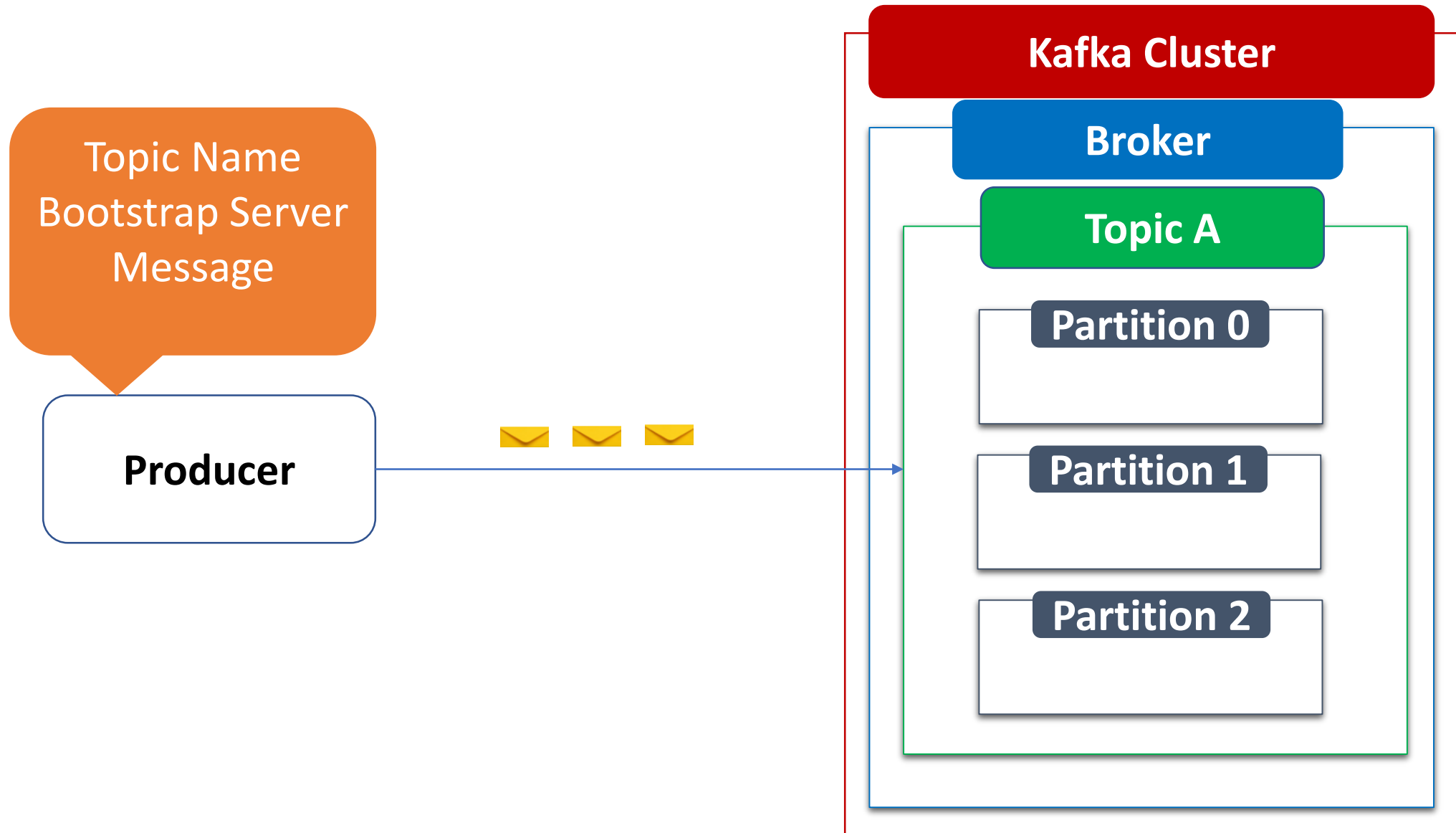
Producer

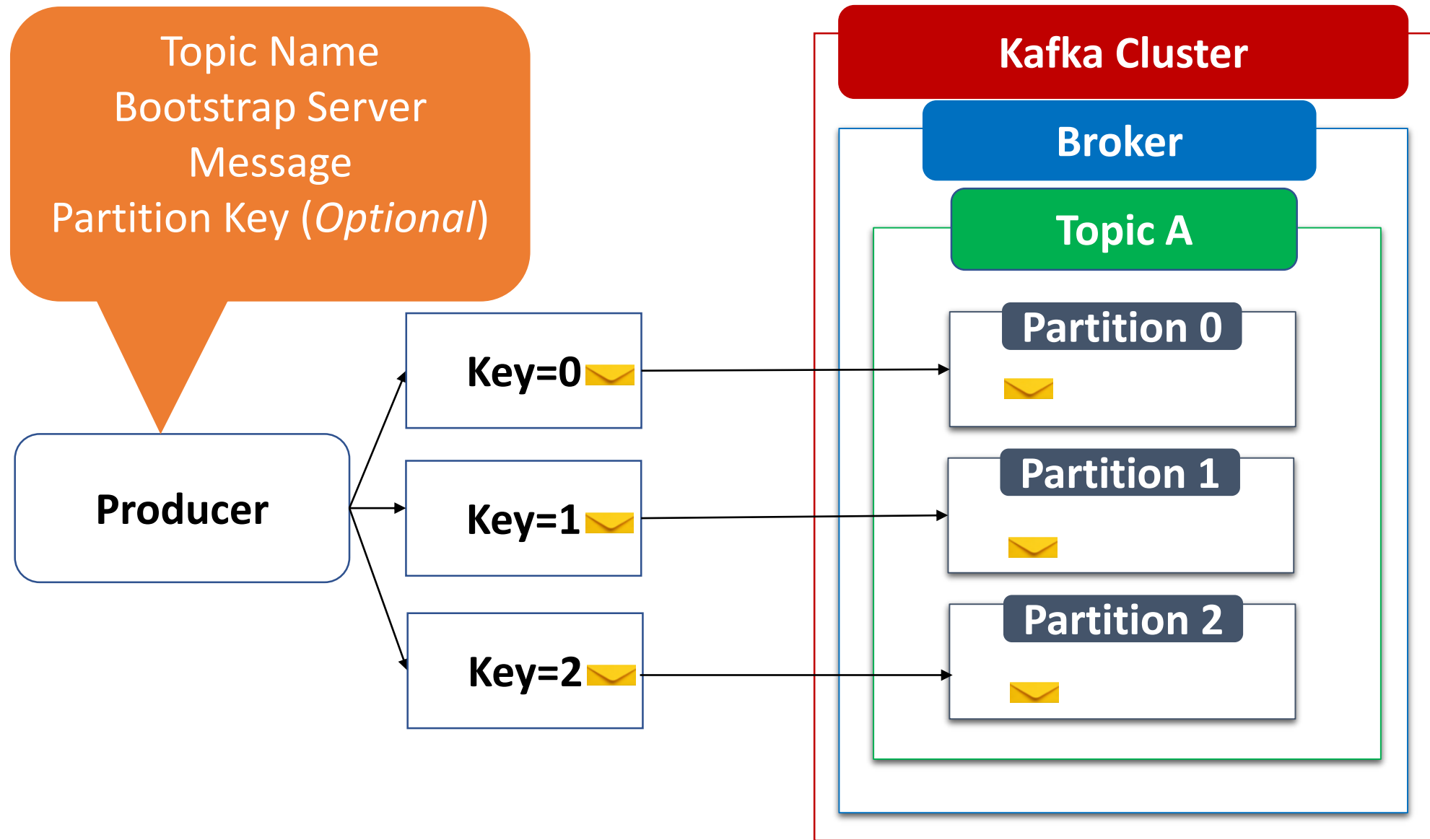
How is a partition determined for a given message?

- It is based on partition key
- Producer can specify the partition key so that messages with same partition key always ends up in the same partition
- Partition key can be of any type (Ex. String, Number etc.)

What if Producer doesn't specify any partition key?

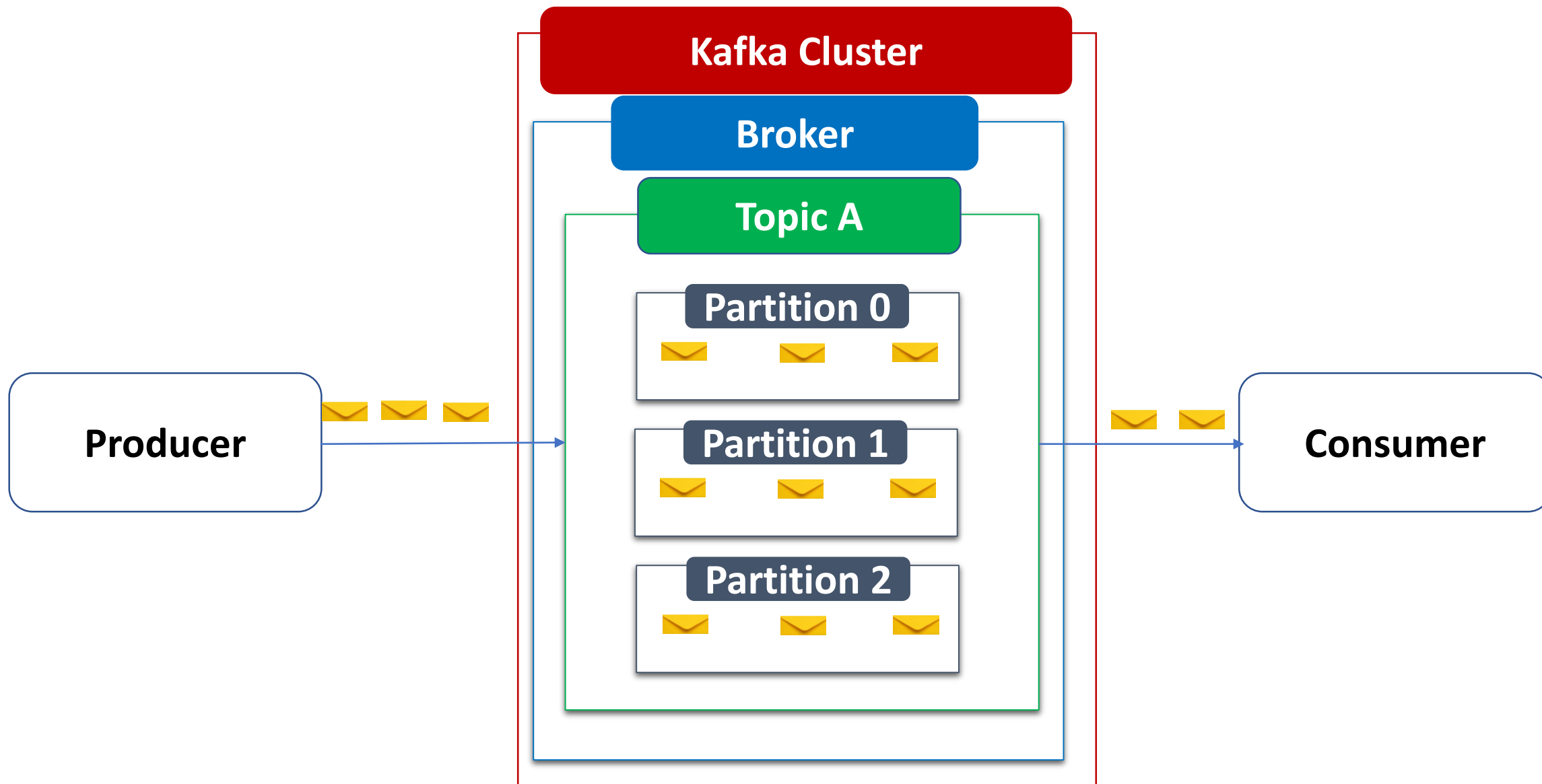
- In that case messages are sent to partitions in round-robin





Consumer

- Consumers read data from a Topic
- Consumers subscribes to the Topic and consume published messages by polling data from the brokers
- Kafka Consumer follows poll model (pulling messages from Broker instead of Broker pushing messages to Consumer)



Consumer Group

- Before going to discuss about Consumer Group let us try to understand a scenario
- Say we have an Application A (Producer) which publishes messages and Application B (Consumer) which process it and store it in database
- In this case, Application B will create a Consumer, subscribes to the appropriate Topic and starts consuming messages

Consumer Group

- Now what if rate of publishing messages from Producer exceeds rate of consuming messages from Consumer?
- Then Consumer lags and it can cause performance issues

*How can we scale up Consumers to
keep up with the increasing load of
messages?*



Through Consumer Groups

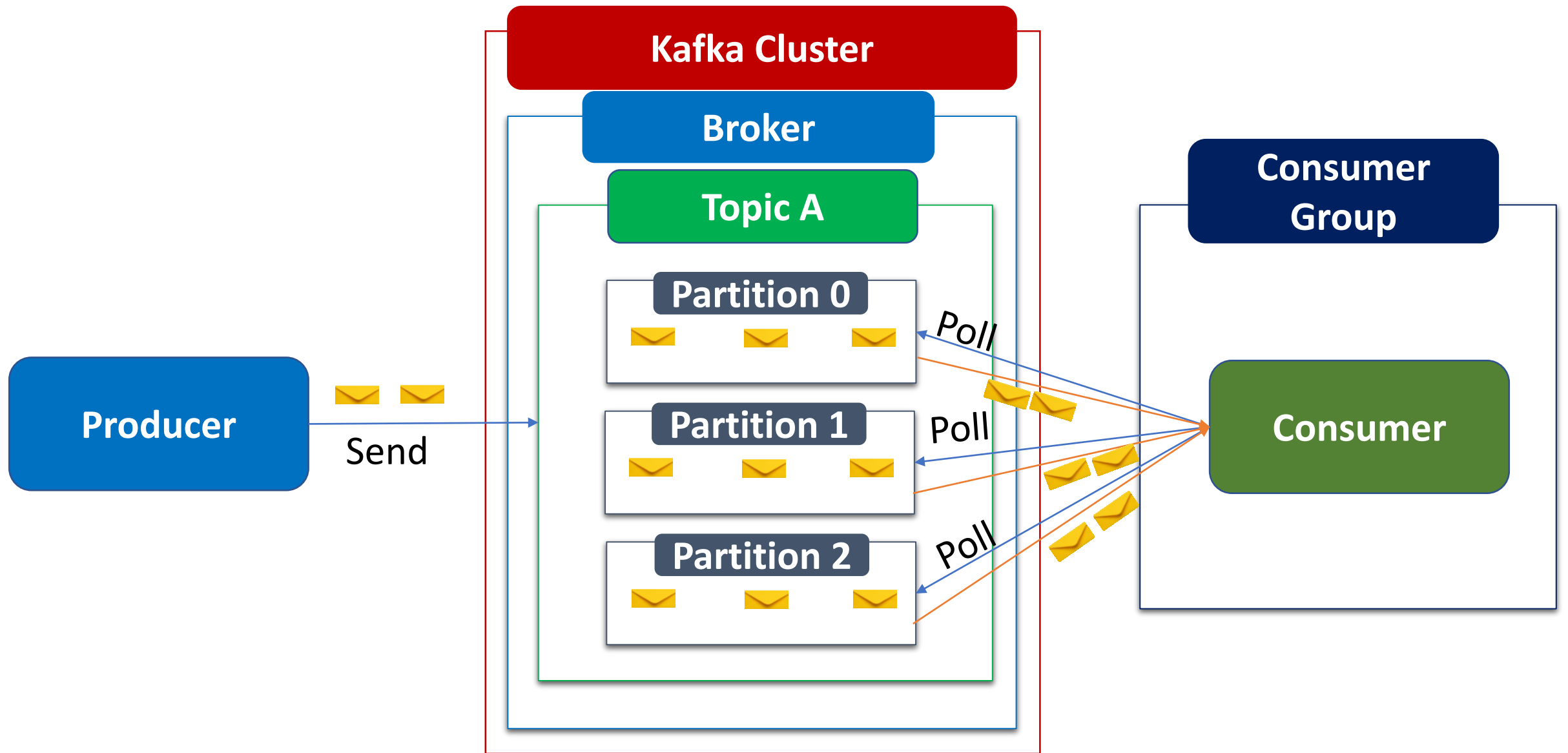


Consumer Group

- In simple terms, Consumer Group is nothing but a group of consumers
- Consumers are part of a Consumer Group
- Consumers belonging to same Consumer Group read data from *mutually exclusive partitions*

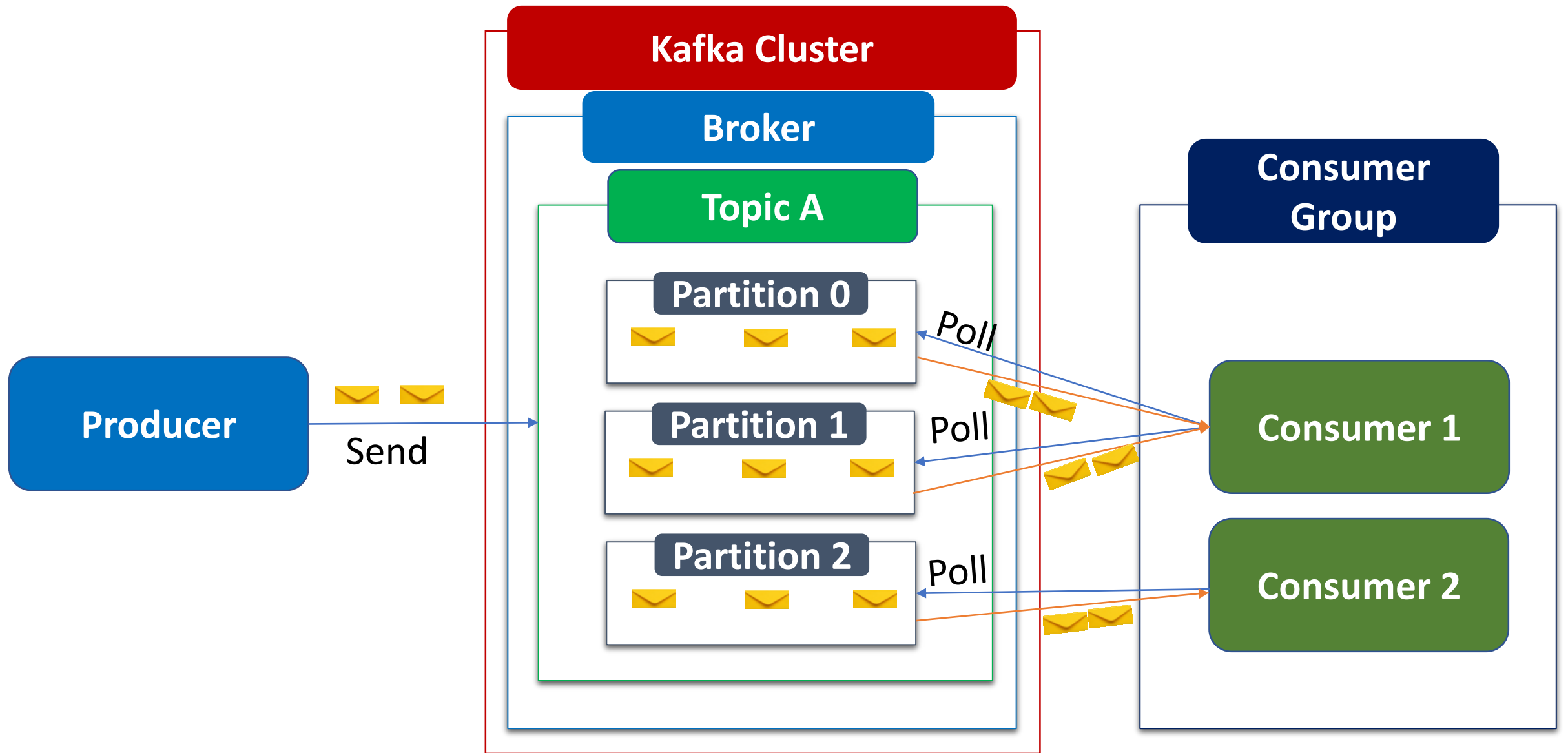
Consumer Group

- Topic A with 3 Partitions
- Consumer Group with 1 Consumer



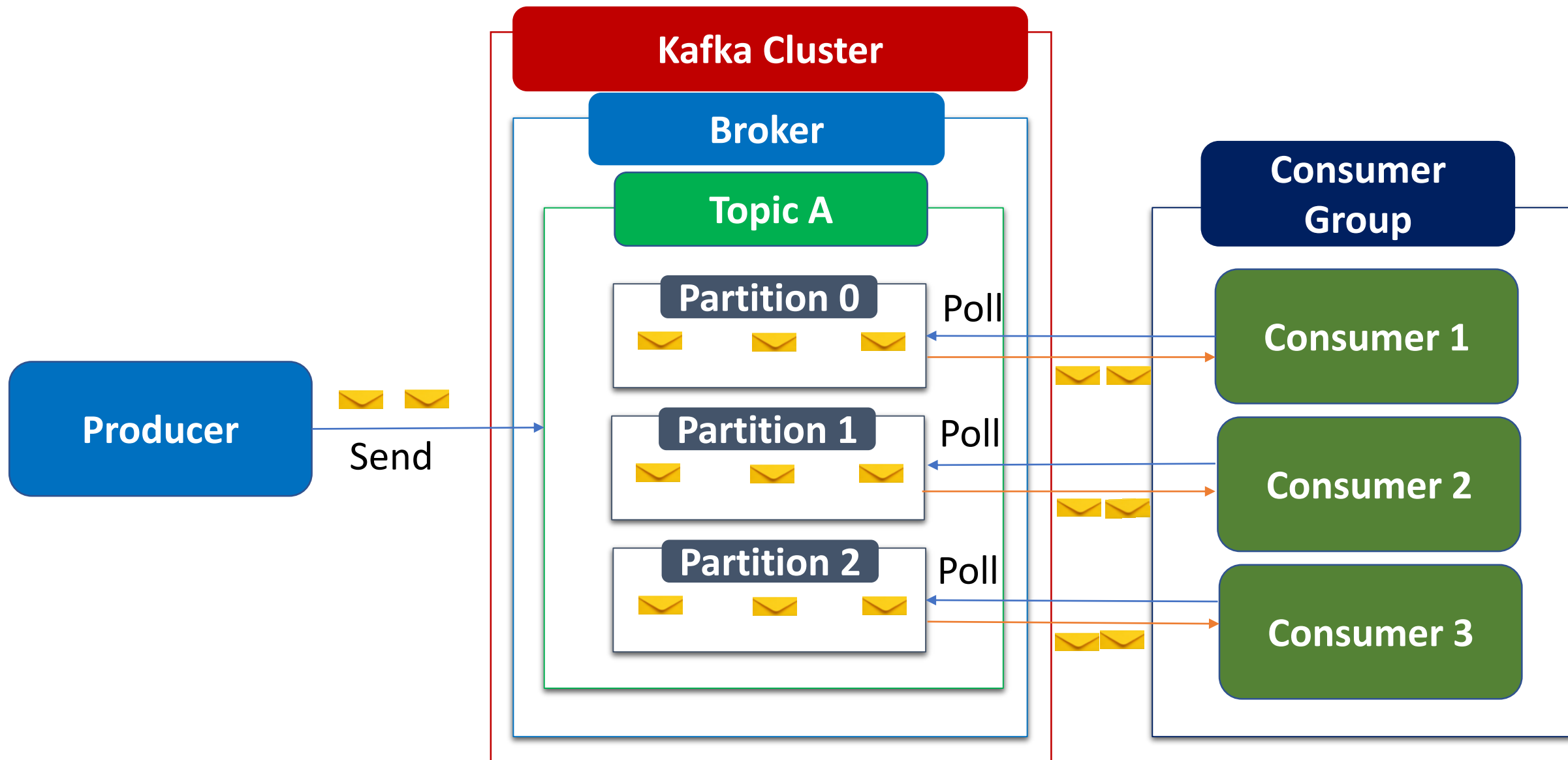
Consumer Group

- Topic A with 3 Partitions
- Consumer Group 1 with 2 Consumers



Consumer Group

- Topic A with 3 Partitions
- Consumer Group 1 with 3 Consumers

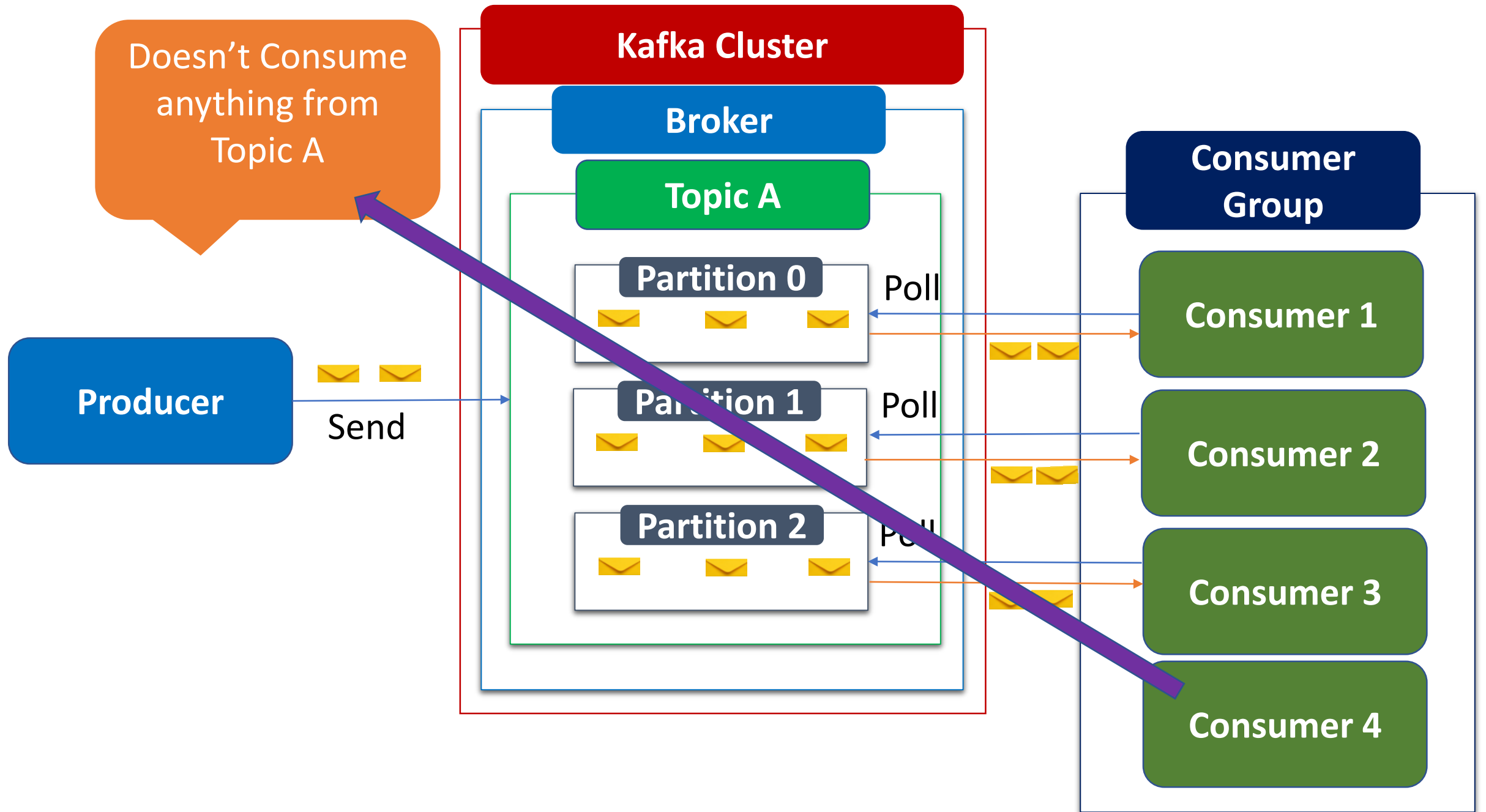


*Can I create unlimited number of
Consumers to scale up?*



Consumer Group

- Topic A with 3 Partitions
- Consumer Group 1 with 4 Consumers

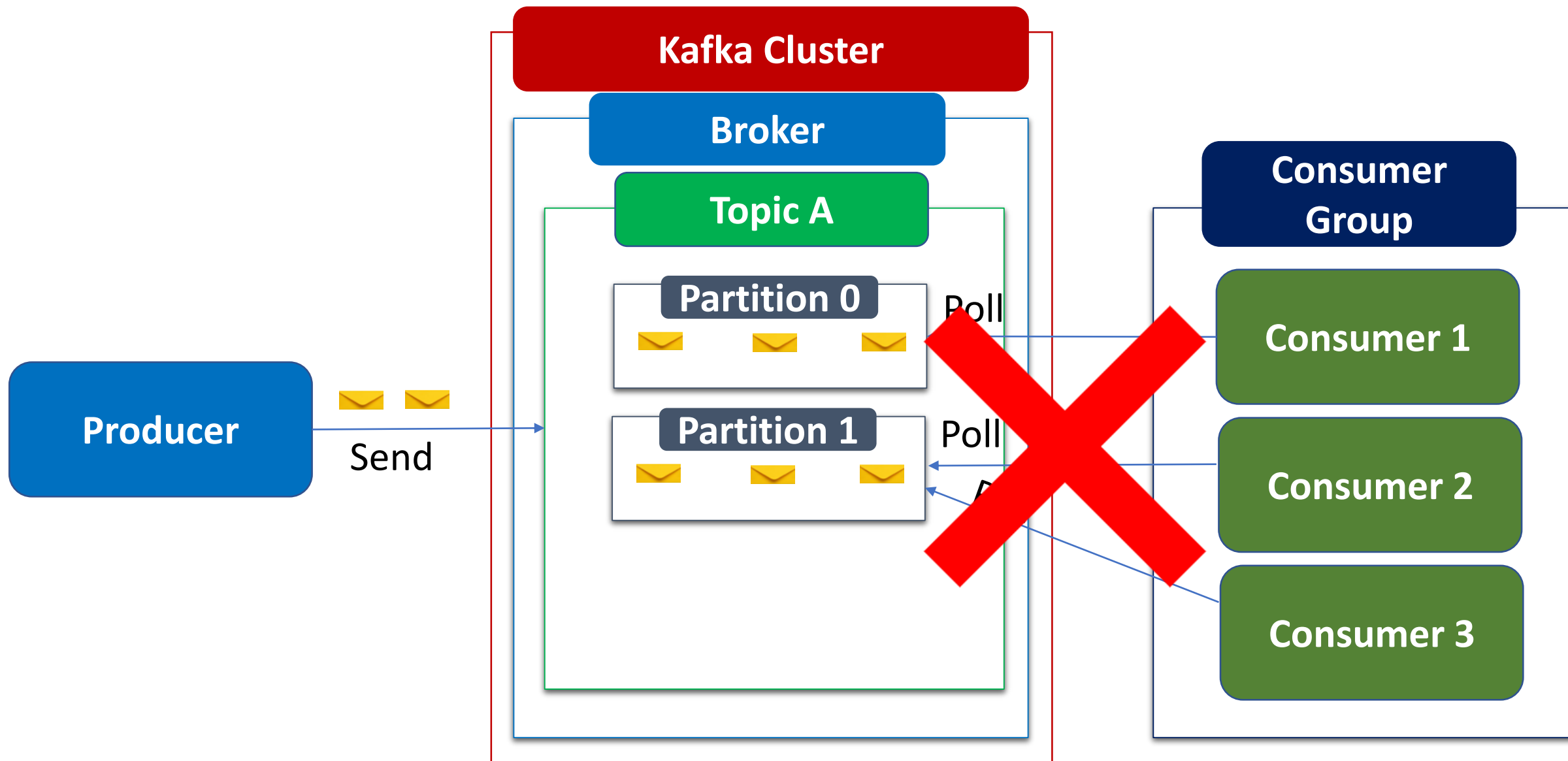


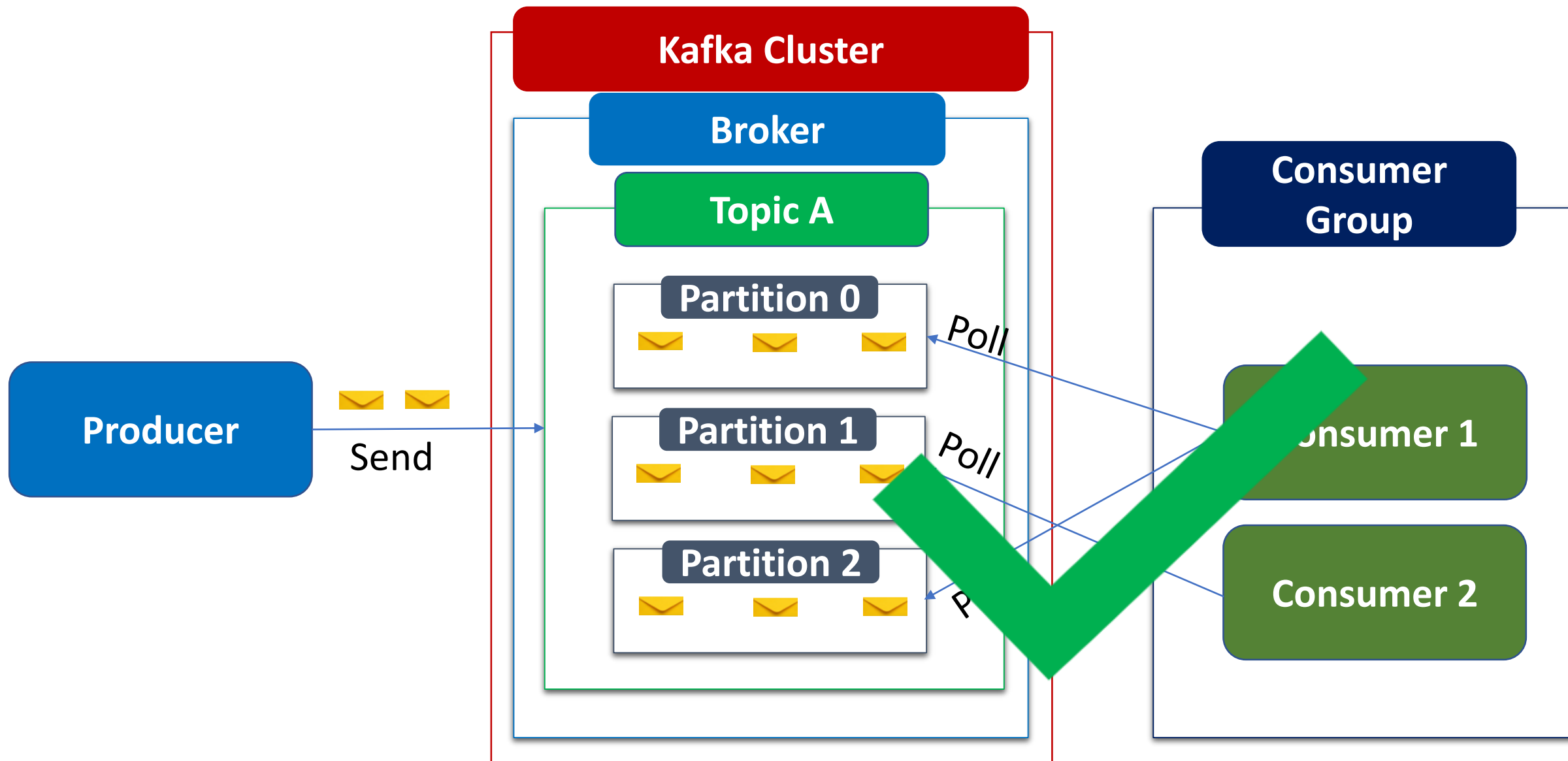
*Why Consumer 4 doesn't consume
from Topic A?*



Consumer Group

- Because a Partition can't be read by multiple Consumers.
- A Partition can be assigned to one and only one Consumer
- But A Consumer can read from multiple Partitions





Consumer Group

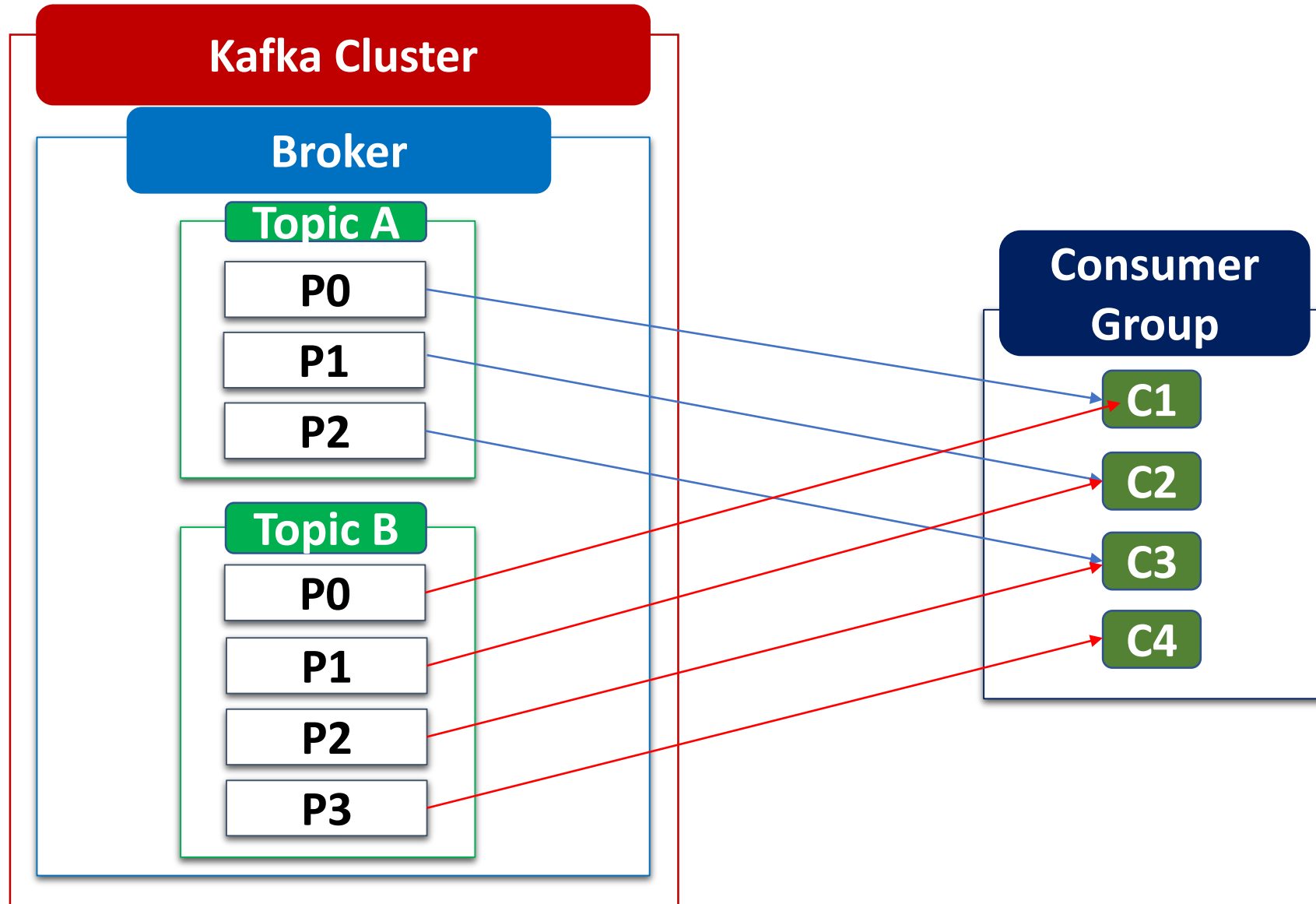
- Because a Partition can't be read by multiple Consumers.
- A Partition can be assigned to one and only one Consumer
- But A Consumer can read from multiple Partitions

*Can Consumer Group read from
multiple Topics?*



Consumer Group

- Topic A with 3 Partitions
- Topic B with 4 Partitions
- Consumer Group 1 with 4 Consumers

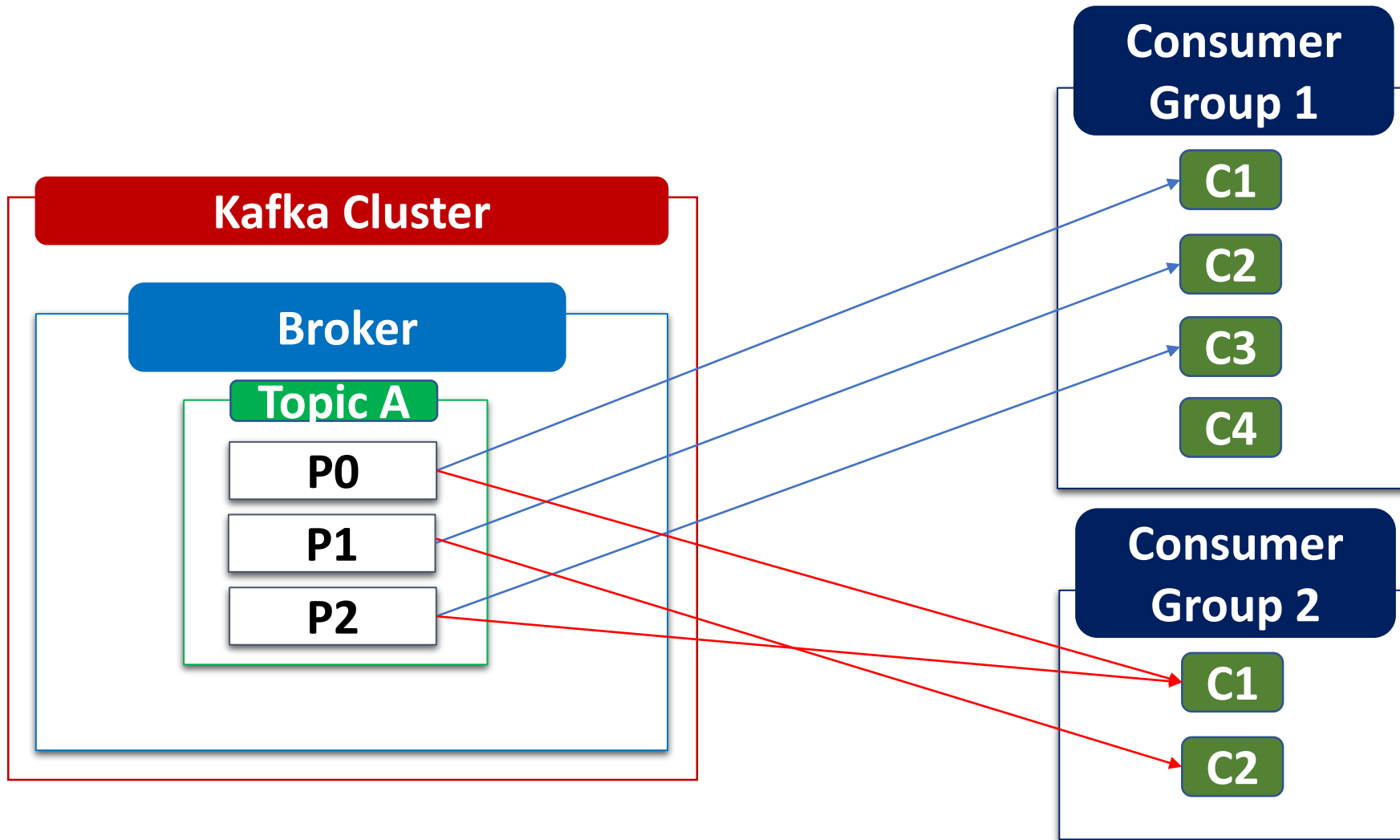


*Can same Topic be read by multiple
Consumer Groups? Yes*



Consumer Group

- Topic A with 3 Partitions
- Consumer Group 1 with 4 Consumers
- Consumer Group 2 with 2 Consumers



Zookeeper

- Apache Zookeeper is an open-source distributed coordination service
- Zookeeper is mandatory for Kafka to work

Zookeeper

- In Production Zookeeper need minimum 3 servers to maintain high availability (It follows Leader-Follower model)

*Role of **Zookeeper** in Kafka*

Manages Kafka Cluster

**Tracks Broker State
(Dead/Alive)**

**Maintains Topics
Metadata**

Replication Needs



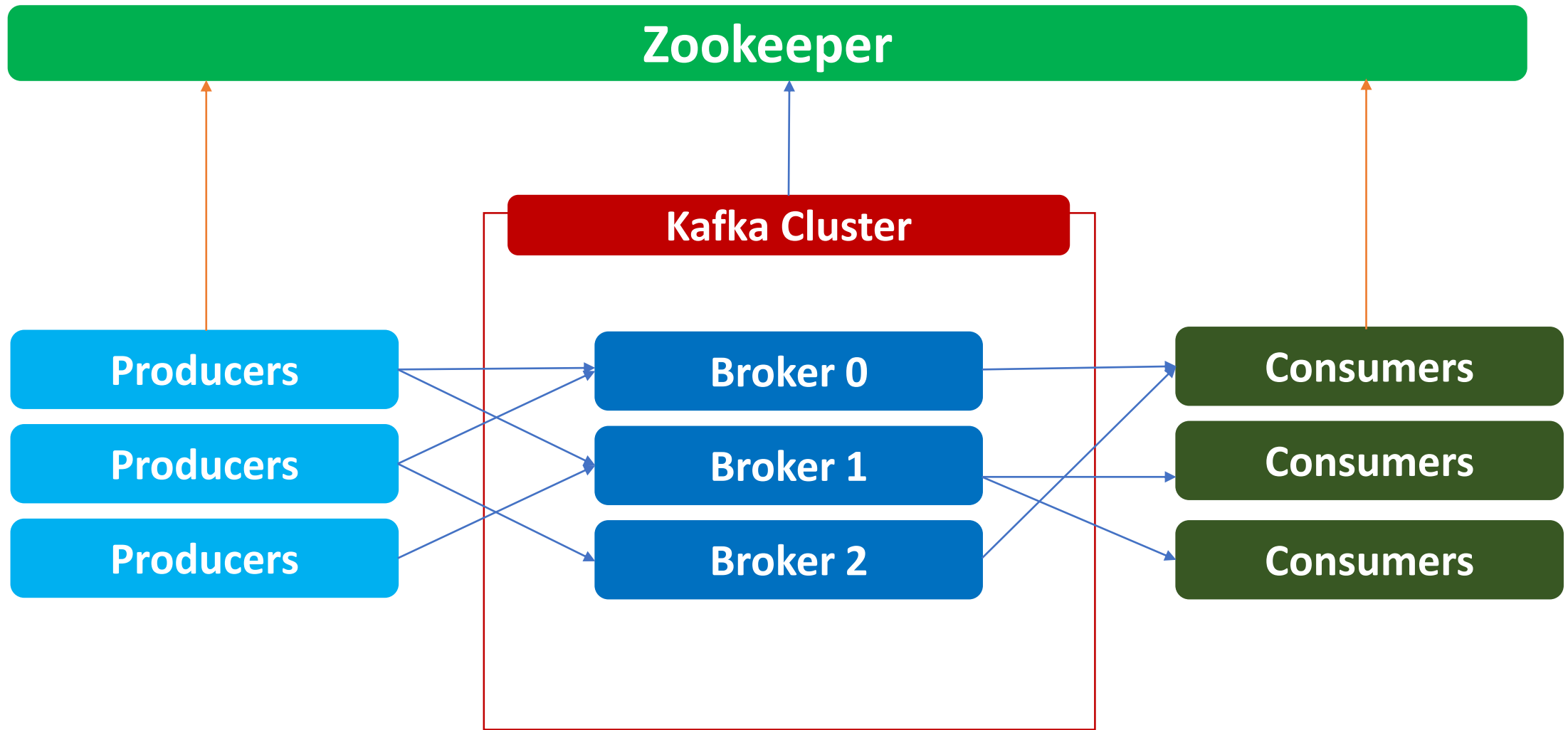
**Maintains Kafka
Cluster Metadata**

**Notifies
Producers/Consumers
about Broker(s) state**

**Helps in Partition
Leader Election**

**Maintains Access
Control Lists (ACL)**

Kafka Architecture



Kafka Key Features

- **Horizontal scalability** -
 - Can scale to multiple brokers
 - Can scale to millions of messages per second
- **Fault tolerant** - Kafka is resistant to node/machine failure within a cluster
- **High Volume** - Kafka can work with the huge volume of data streams, easily.
- **High performance** – Kafka can support message throughput of thousands of messages per second

Kafka Key Features

- **Durability** - Refers to the persistence of data/messages on disk. Also, messages replication is one of the reasons behind durability, hence messages are never lost
- **Distributed** - The distributed architecture of Kafka makes it scalable using capabilities like replication and partitioning
- **Variety of use cases** – It can cater to variety of use cases like Messaging system, Website activity tracking, Log aggregation, Metrics collection etc.)
- **Decouples** system dependencies

Summary

- What is Kafka?
- Kafka cluster
- Broker
- Producer
- Consumer
- Consumer Groups
- Zookeeper
- Kafka features

Kafka setup

- Install Java & Kafka
- Check installation
- Edit path variable
- Create data directories
- Edit Zookeeper configuration
- Edit Kafka configuration
- Start Zookeeper service
- Start Kafka service

Kafka Command Line Interface

Kafka Command Line Interface

Topic related commands

Kafka Command Line Interface

Create Topic

Kafka Command Line Interface

- Command to create a topic

```
kafka-topics --zookeeper localhost:2181  
              --topic order_info_topic --create  
              --partitions 1
```

Kafka Command Line Interface

Create Topic Demo

Kafka Command Line Interface

List Topics

Kafka Command Line Interface

- Command to list topics

```
kafka-topics --zookeeper localhost:2181  
--list
```

Kafka Command Line Interface

List Topics Demo

Kafka Command Line Interface

Describe A Topic

Kafka Command Line Interface

- Command to describe a topic

```
kafka-topics --zookeeper localhost:2181  
              --topic order_info_topic  
              --describe
```

Kafka Command Line Interface

Describe A Topic Demo

Kafka Command Line Interface

Delete A Topic

Kafka Command Line Interface

- Command to delete a topic

```
kafka-topics --zookeeper localhost:2181  
              --topic order_info_topic  
              --delete
```

Kafka Command Line Interface

Producer related commands

Kafka Command Line Interface

Publish Messages To A Topic

Kafka Command Line Interface

- Command to produce data to a topic

```
kafka-console-producer --broker-list localhost:9092  
--topic order_info_topic
```

Kafka Command Line Interface

Publish Messages To A Topic Demo

Kafka Command Line Interface

Publish Messages To A Topic Demo

Kafka Command Line Interface

Consumer related commands

Kafka Command Line Interface

Consuming Messages From A Topic

Kafka Command Line Interface

- Command to consume data from a topic

```
kafka-console-consumer --bootstrap-server localhost:9092  
                        --topic order_info_topic
```

Kafka Command Line Interface

Consuming Messages From A Topic Demo

Kafka Command Line Interface

Consuming Messages From Beginning

Kafka Command Line Interface

- Command to consume data from beginning of a topic

```
kafka-console-consumer --bootstrap-server localhost:9092  
                        --topic order_info_topic  
                        -- from-beginning
```

Kafka Command Line Interface

Consuming Messages From Beginning Demo

Kafka Command Line Interface

Consuming Messages From Specific Offset

Kafka Command Line Interface

- Command to consume data from a topic

```
kafka-console-consumer --bootstrap-server localhost:9092  
                        --topic order_info_topic  
                        -- partition 0  
                        -- offset earliest/latest/positive number
```

Kafka Command Line Interface

Consuming Messages From Specific Offset Demo

Kafka Command Line Interface

Consumer Group related commands

Kafka Command Line Interface

- Command to create consumer with Consumer Group

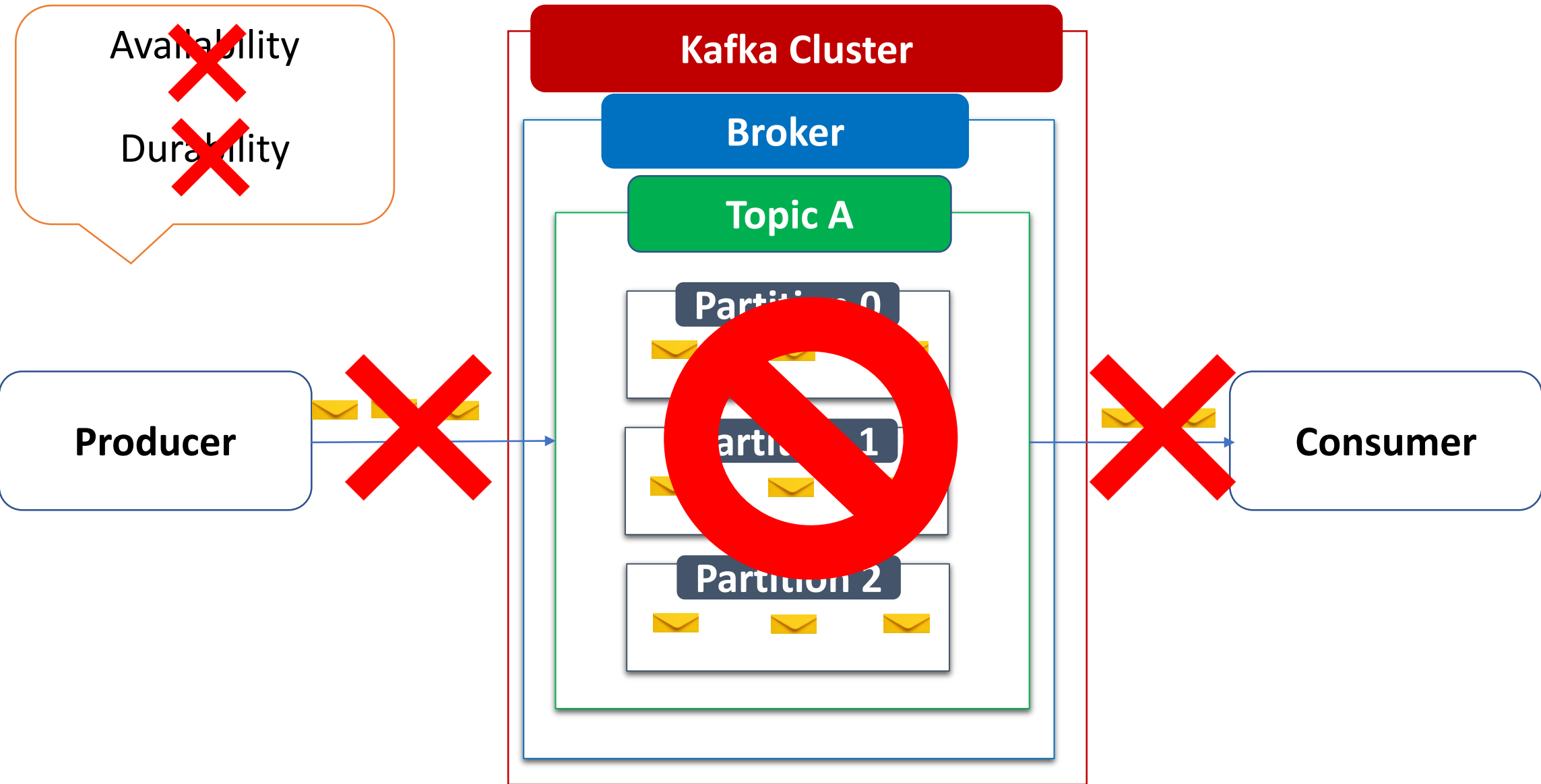
```
kafka-console-consumer --bootstrap-server localhost:9092  
                        --topic assortment_info_topic  
                        --group bv_consumer_group
```

Kafka Command Line Interface

Consumer Groups Demo

What happens if the broker goes down?



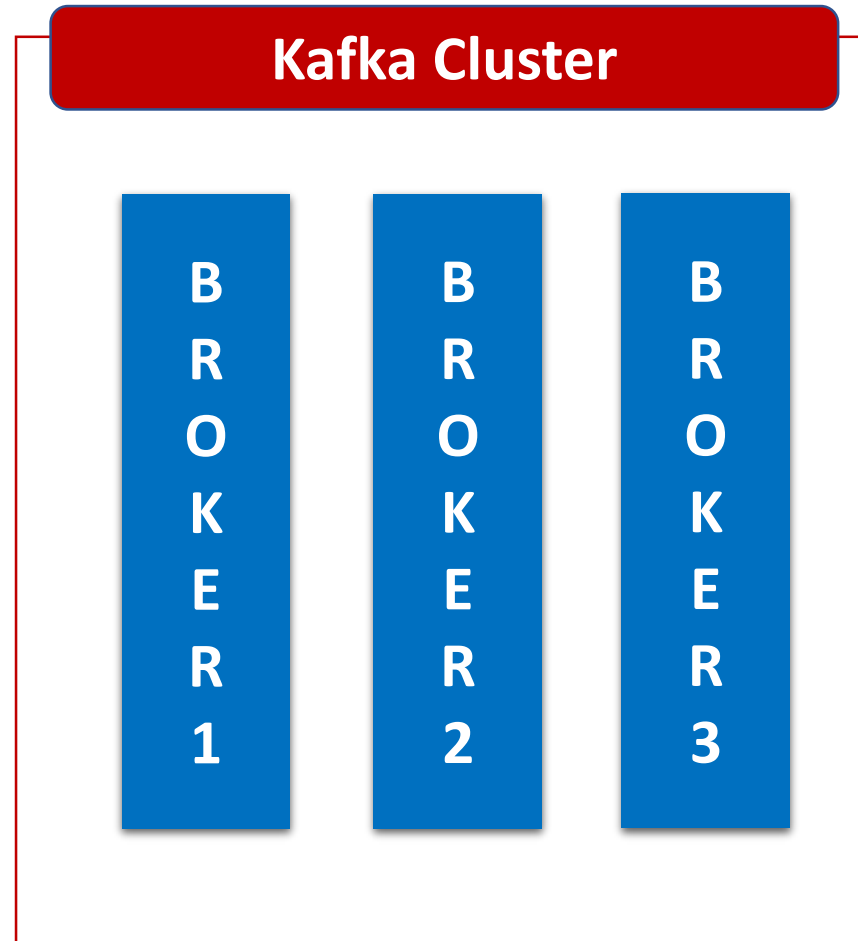


Availability through **Kafka Cluster**
Durability through **Replication Factor**



Kafka Cluster

Kafka Cluster



Kafka Cluster Setup

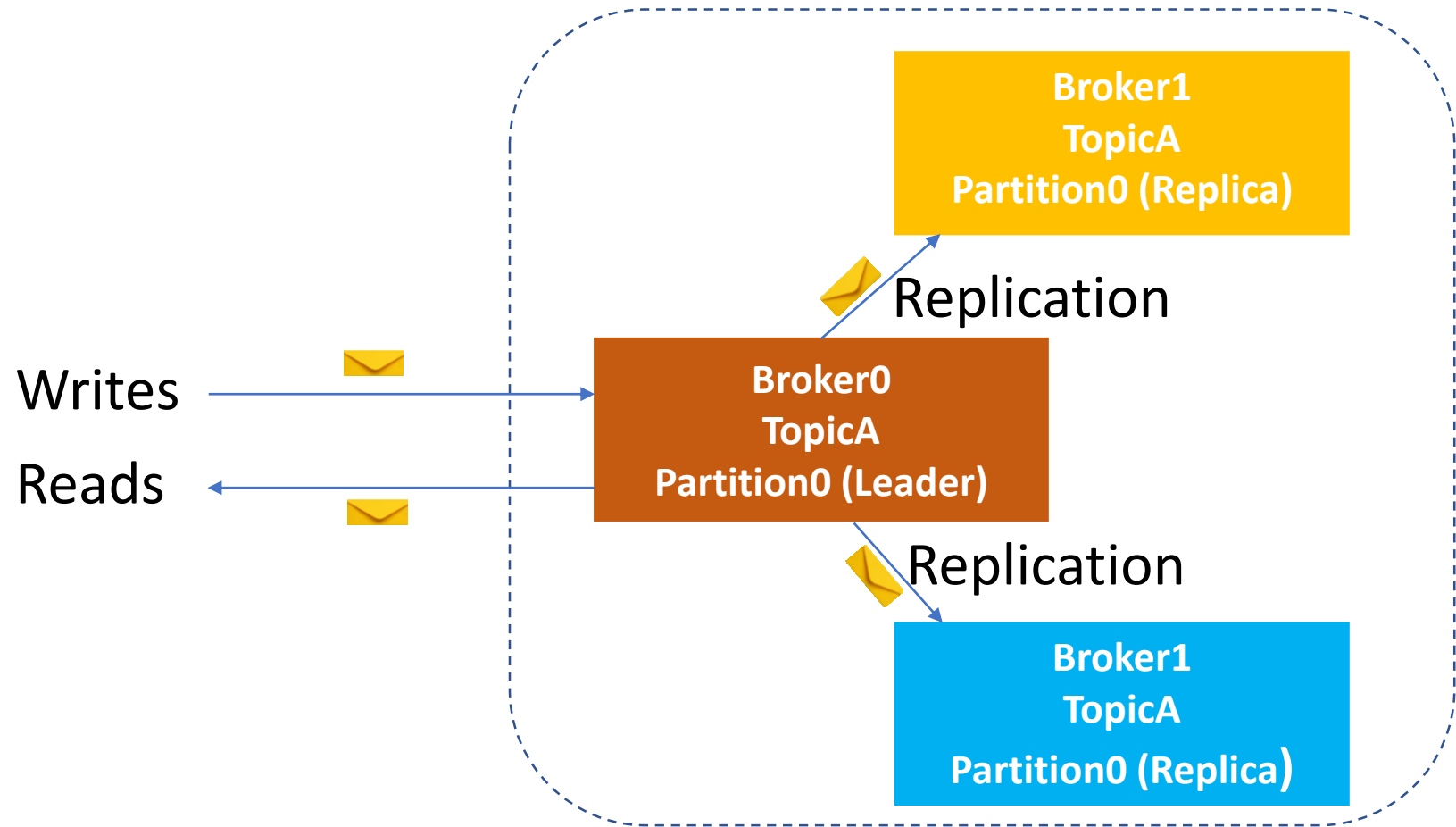
Kafka Cluster Setup Demo

Replication

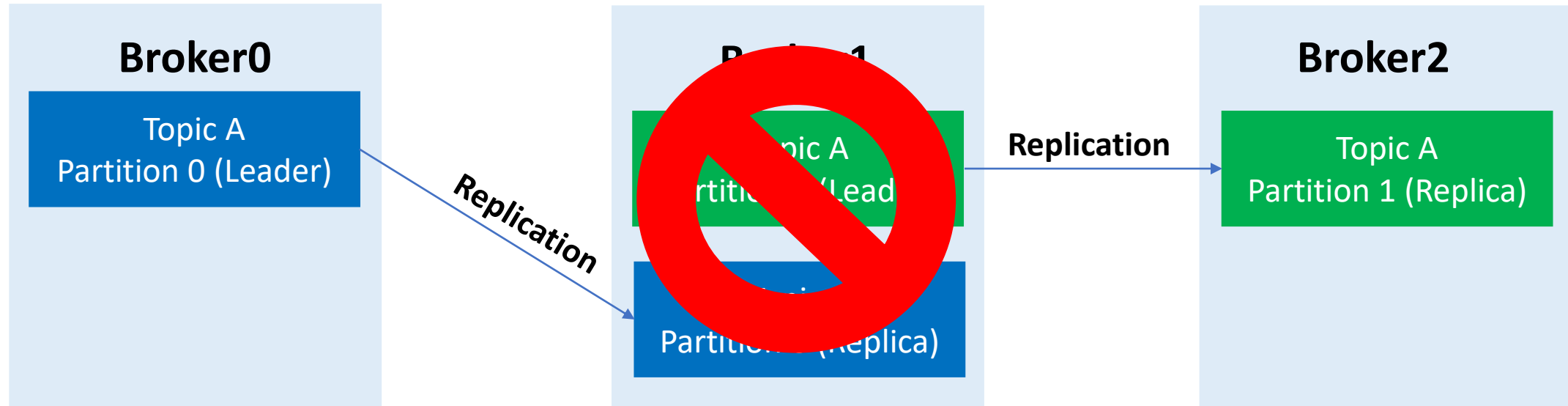
Replication

- Topics should have replication factor > 1
- For a given partition one Broker acts as a Leader. All the produce & consume requests go through it
- All the remaining replicas of this partition are called followers. They simply replicate messages from the leader. They don't serve produce & consume requests
- With replication in place, if a broker goes down, partition data can be served from another broker

Replication



Replication Factor = 2 Example



Kafka setup

Kafka setup : Install Java & Kafka

- Java
- Kafka (<https://kafka.apache.org/downloads>)

Kafka setup : Check installation

- Once installed unzip Kafka and copy the folder in C:\
- Check java installed or not using command – java -version

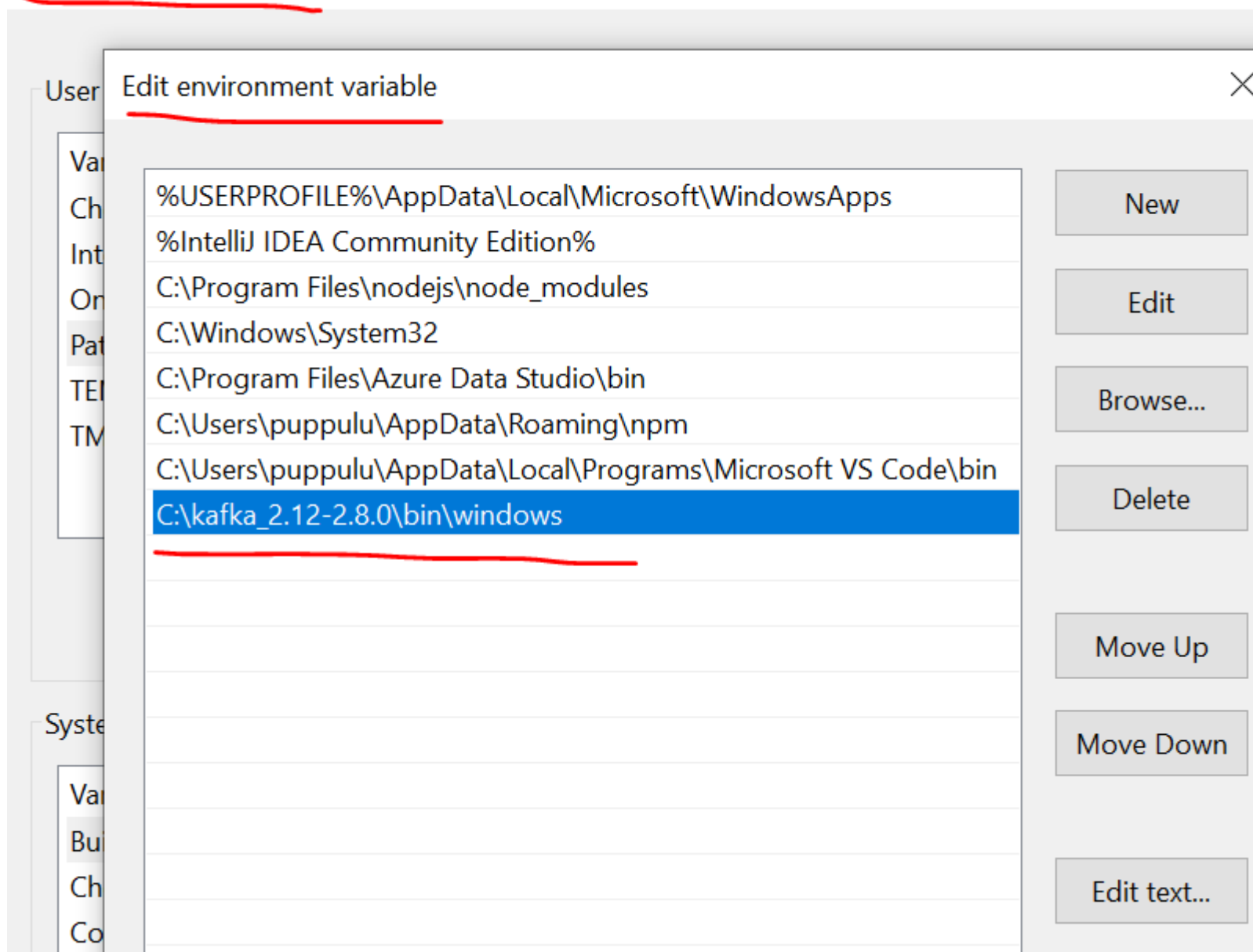
```
C:\>java -version
java version "1.8.0_281"
Java(TM) SE Runtime Environment (build 1.8.0_281-b09)
Java HotSpot(TM) Client VM (build 25.281-b09, mixed mode)
```

- Now go to C:\kafka_2.12-2.8.0\bin\windows
- Run command kafka-topics.bat – It should run with out errors

Kafka setup : Edit path variable

- Now set PATH variable to C:\kafka_2.12-2.8.0\bin\windows

Environment Variables



Kafka setup : Create data directories

- Create data directories for Zookeeper & Kafka
- Create directory data under C:\kafka_2.12-2.8.0
- Under data directory create directories with names zookeeper, kafka

```
C:\kafka_2.12-2.8.0\data>dir
Volume in drive C is OSDisk
Volume Serial Number is 2C82-E4D9

Directory of C:\kafka_2.12-2.8.0\data

06/28/2021  11:00 AM    <DIR>          .
06/28/2021  11:00 AM    <DIR>          ..
06/28/2021  11:00 AM    <DIR>          kafka
06/28/2021  11:00 AM    <DIR>          zookeeper
               0 File(s)                0 bytes
               4 Dir(s)  409,695,756,288 bytes free
```

Kafka setup : Edit Zookeeper configuration

- Now change Zookeeper data configuration property
- Open zookeeper.properties under C:\kafka_2.12-2.8.0\config
- Now change the data directory as below
- dataDir=C:/kafka_2.12-2.0.0/data/zookeeper

Kafka setup : Edit Kafka configuration

- Now change Kafka data configuration property
- Open server.properties under C:\kafka_2.12-2.8.0\config
- Now change the data directory as below
- `log.dirs=C:/kafka_2.12-2.8.0/data/kafka`

Kafka setup : Start Zookeeper service

- Run below commands to start Zookeeper
 - cd C:/kafka_2.12-2.8.0
 - Run command **zookeeper-server-start.bat config/zookeeper.properties**

```
C:\kafka_2.12-2.8.0>zookeeper-server-start.bat config/zookeeper.properties
```

```
[2021-06-28 11:19:41,166] INFO Using org.apache.zookeeper.server.NIOServerCnxnFactory as server connection factory (org.apac
[2021-06-28 11:19:41,170] INFO Configuring NIO connection handler with 10s sessionless connection timeout, 2 selector thread
. (org.apache.zookeeper.server.NIOServerCnxnFactory)
[2021-06-28 11:19:41,177] INFO binding to port 0.0.0.0/0.0.0.0:2181 (org.apache.zookeeper.server.NIOServerCnxnFactory)
[2021-06-28 11:19:41,191] INFO zookeeper.snapshotSizeFactor = 0.33 (org.apache.zookeeper.server.ZKDatabase)
[2021-06-28 11:19:41,195] INFO Snapshotting: 0x0 to C:\kafka_2.12-2.8.0\data\zookeeper\version-2\snapshot.0 (org.apache.zook
[2021-06-28 11:19:41,200] INFO Snapshotting: 0x0 to C:\kafka_2.12-2.8.0\data\zookeeper\version-2\snapshot.0 (org.apache.zook
[2021-06-28 11:19:41,210] INFO PrepRequestProcessor (sid:0) started, reconfigEnabled=false (org.apache.zookeeper.server.Prep
[2021-06-28 11:19:41,214] INFO Using checkIntervalMs=60000 maxPerMinute=10000 (org.apache.zookeeper.server.ContainerManager)
```

Kafka setup : Start Kafka service

- Run below commands to start Kafka
 - cd C:/kafka_2.12-2.0.0
 - Run command **kafka-server-start.bat config/server.properties**

```
C:\kafka_2.12-2.0.0>kafka-server-start.bat config/server.properties
```

```
[2021-06-28 11:51:34,569] INFO [SocketServer listenerType=ZK_BROKER, nodeId=0] Started socket server acceptors
[2021-06-28 11:51:34,572] INFO Kafka version: 2.8.0 (org.apache.kafka.common.utils.AppInfoParser)
[2021-06-28 11:51:34,573] INFO Kafka commitId: ebb1d6e21cc92130 (org.apache.kafka.common.utils.AppInfoParser)
[2021-06-28 11:51:34,574] INFO Kafka startTimeMs: 1624861294569 (org.apache.kafka.common.utils.AppInfoParser)
[2021-06-28 11:51:34,585] INFO [KafkaServer id=0] started (kafka.server.KafkaServer)
```

Kafka Java

Producer & Consumer Demo

Producer with Partition Key Demo

Consumer Group Demo

Producer Callback Demo

Broker Failover Demo

Producer Delivery Semantics

Producer Delivery Semantics

- Atmost once
- Atleast once
- Exactly once

How to achieve these delivery semantics?



Using Producer configurations -

- acks
- min.insync.replicas
- retries
- max.in.flight.requests.per.connection
- delivery.timeout.ms



Producer Acknowledgement (acks=0)

- We can get Atmost once guarantee with acks=0
- Producer sends record to Broker and doesn't wait for any response from Broker
- Messages willn't be retried
- It is - send and forget approach
- Chances of data loss - High
- Usecases – Logging, Metric collection etc.

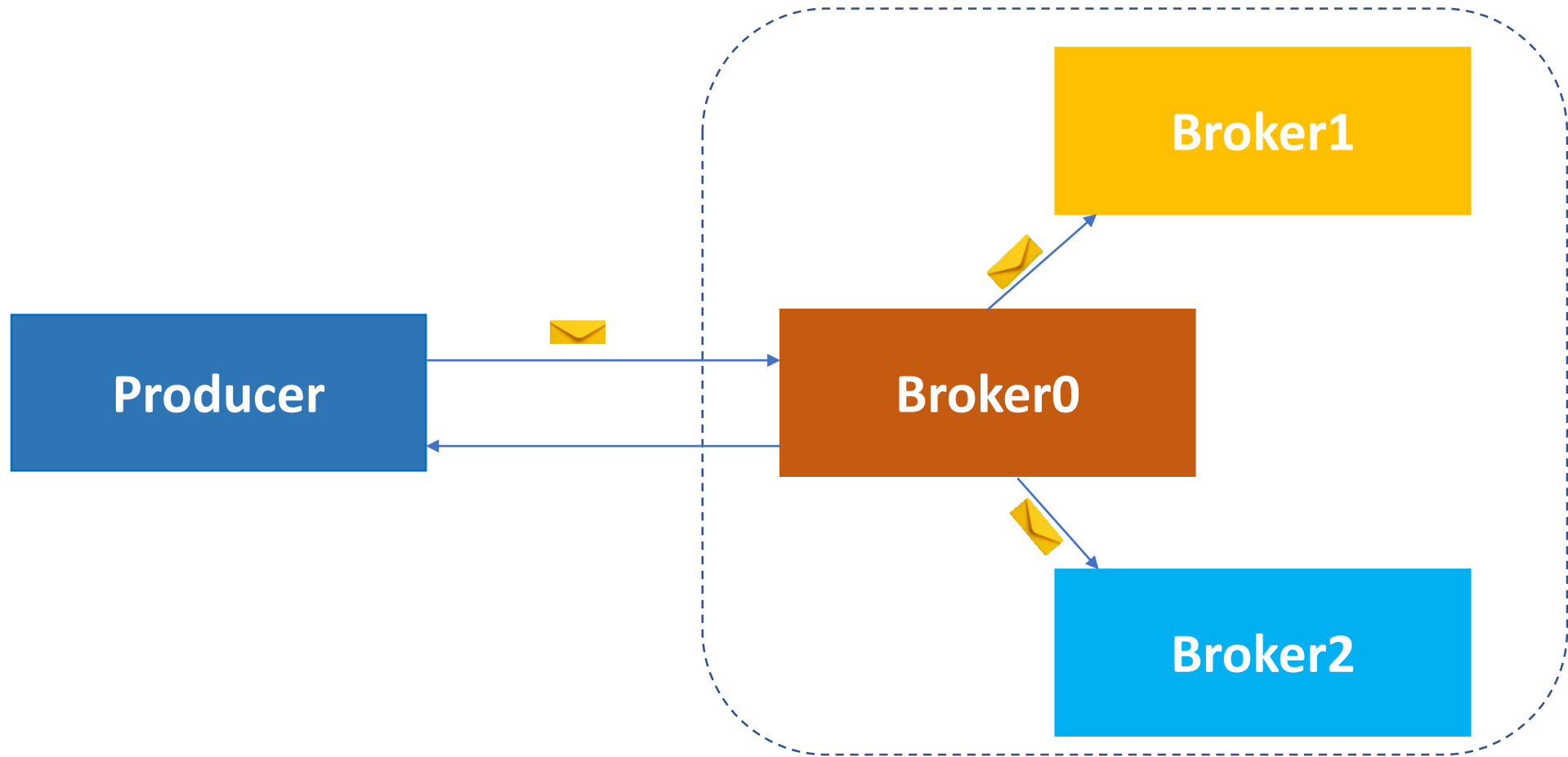
Producer Acknowledgement (acks=1)

- We can get Atleast once guarantee with acks=1
- Producer sends record to Broker and waits for confirmation from Broker
- If no acknowledgement is received, then Producer retries based on retry configuration
- Chances of data loss - Moderate

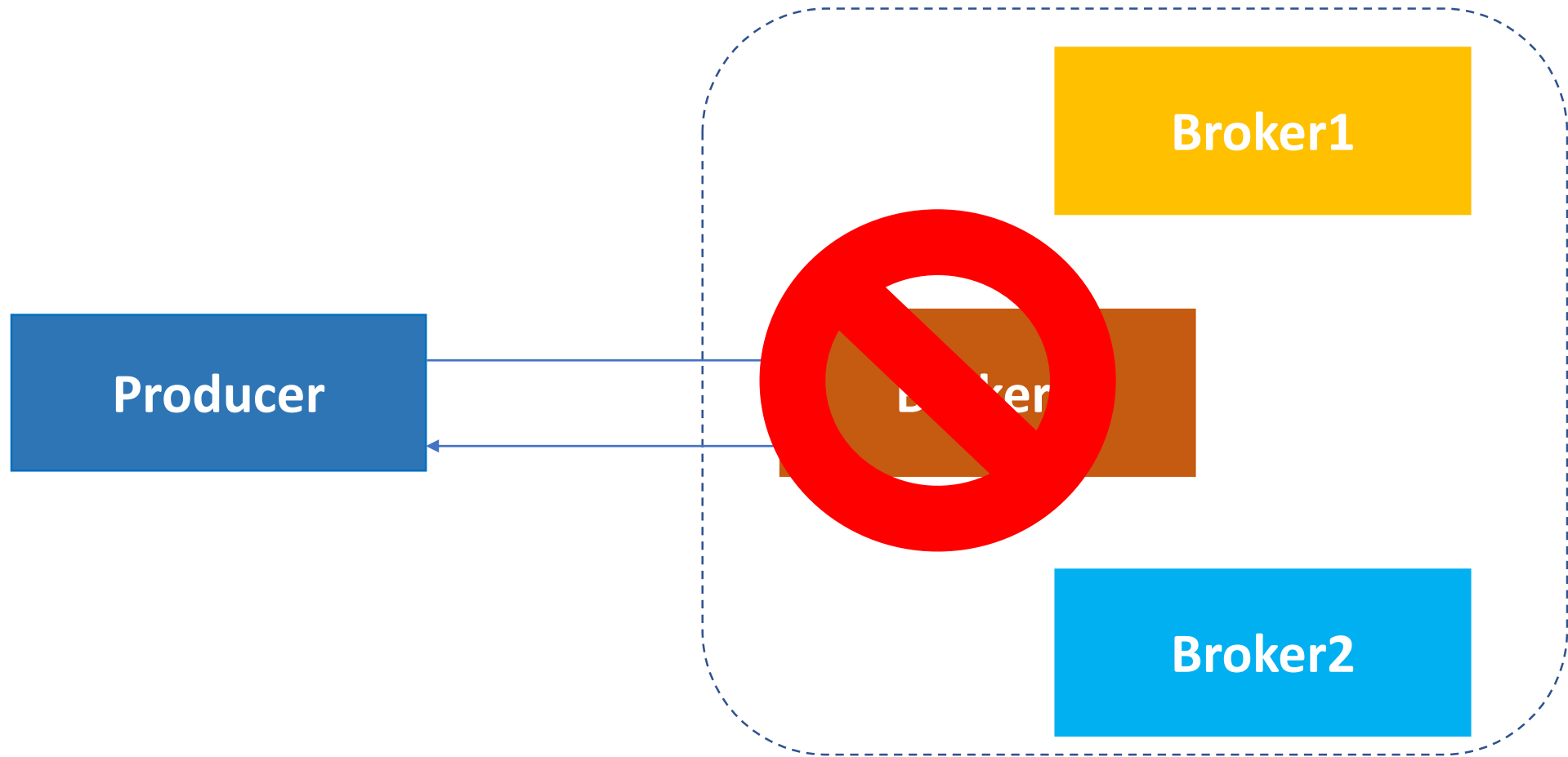
Why moderate data loss though we get acknowledgement from Broker?



Producer Acknowledgement (acks=1)



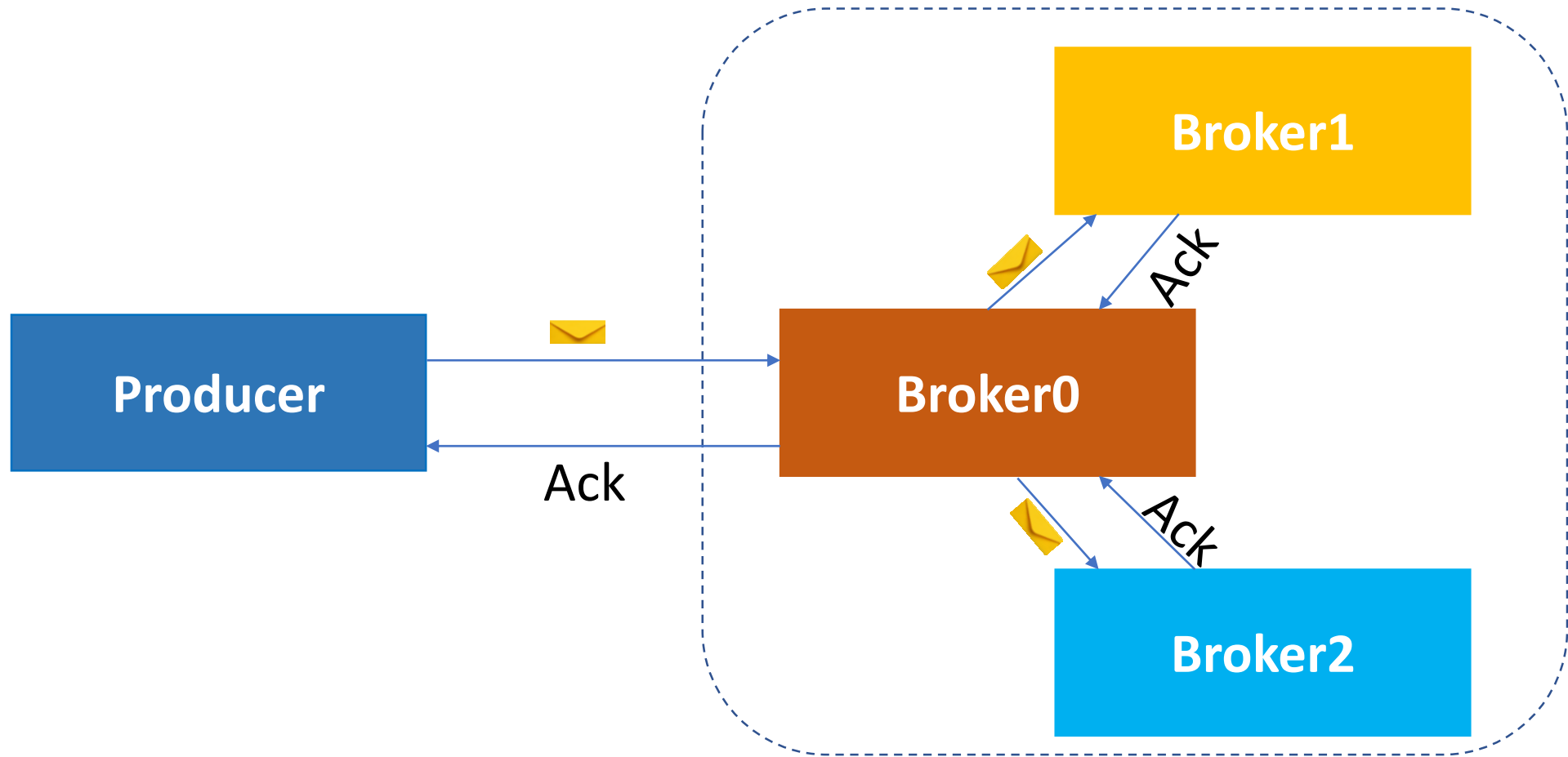
Producer Acknowledgement (acks=1)



Producer Acknowledgement (acks=all)

- We can get Exactly once guarantee with acks=all
- Producer sends record to Broker and waits for confirmation from Broker
- If no acknowledgement is received, then Producer retries based on retry configuration
- Broker sends acknowledgement only after replication
- Chances of data loss - Minimal

Producer Acknowledgement (acks=all)



If we have multiple brokers does the message need to be replicated on all before acknowledgement?



- It depends on **min.insync.replicas** configuration
- If Kafka cluster has 3 Brokers and min.insync.replicas=2 then Broker sends Acknowledgement after the completion of 2nd replication
- min.insync.replicas=2 means minimum 2 brokers (including Leader) must respond that they have data



Producer Acknowledgement (acks=all)

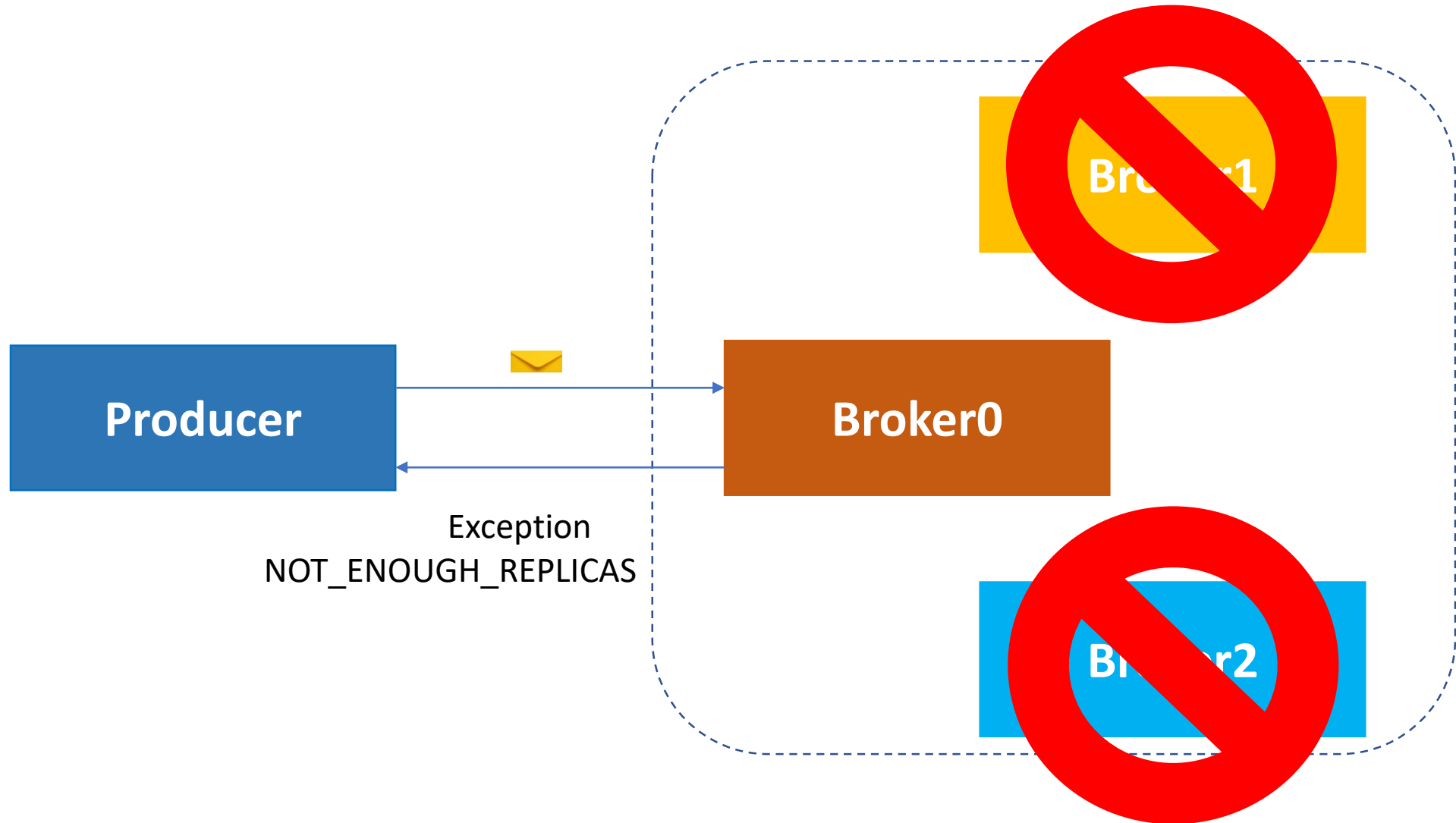
- **Acks=all** should be used along with **min.insync.replicas**
- Advantage – No or minimal data loss
- Disdvantage - Latency

Producer Acknowledgement (acks=all)

- What happens if there are not enough brokers to replicate?

Then the broker will return an exception instead of an acknowledgment.

min.insync.replicas=2



Producer Retries

How many times Producer retries?



Producer Retries

- **request.timeout.ms** - maximum amount of time the client will wait for the response of a request. If the response is not received before the timeout elapses the client will resend the request if necessary or fail the request if retries are exhausted
- **retries** – Number of times the producer will retry sending the message

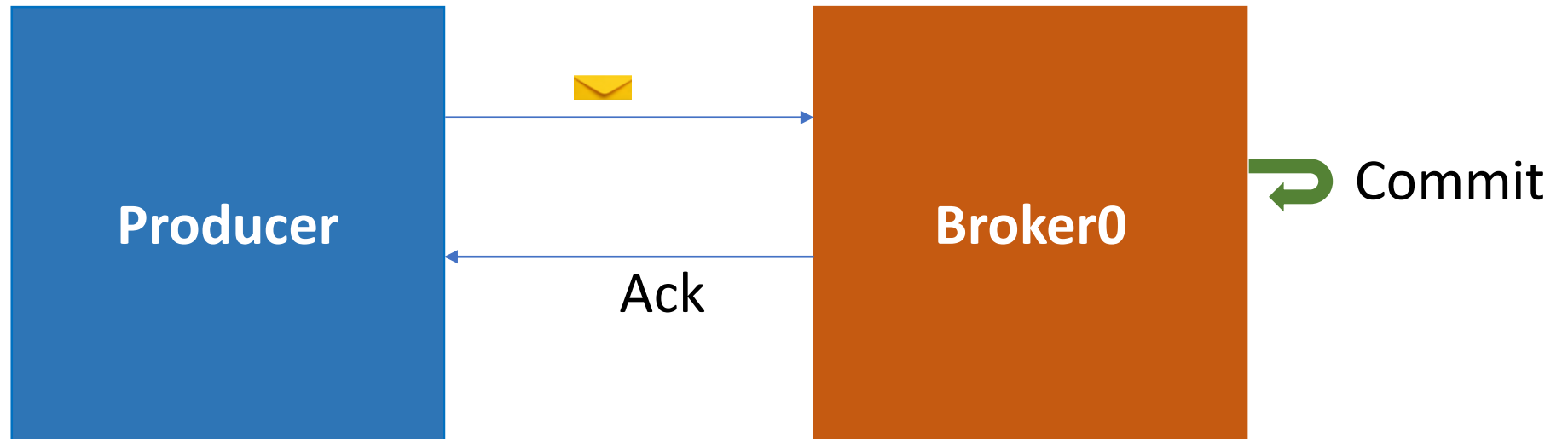
Producer Retries

- **retry.backoff.ms** - The amount of time to wait before attempting to retry a failed request to a given topic partition
- **delivery.timeout.ms** –
 - The time within which producer's send() method either report success/failure
 - The value of this config should be greater than or equal to the sum of *request.timeout.ms* and *linger.ms* (default 0)

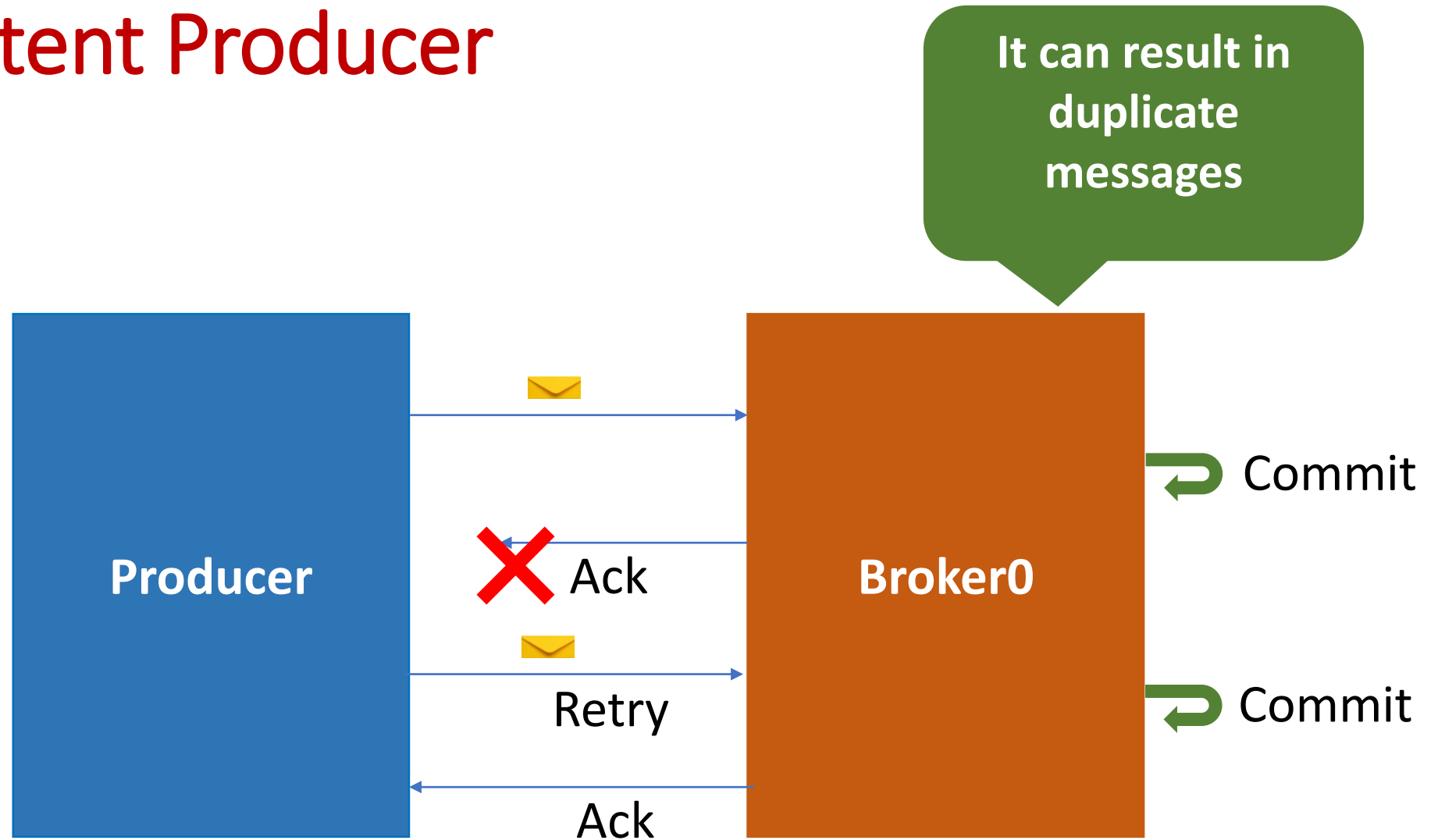
It is Producer's responsibility to capture these failures and do further processing depending on business requirement

Idempotent Producer

Idempotent Producer



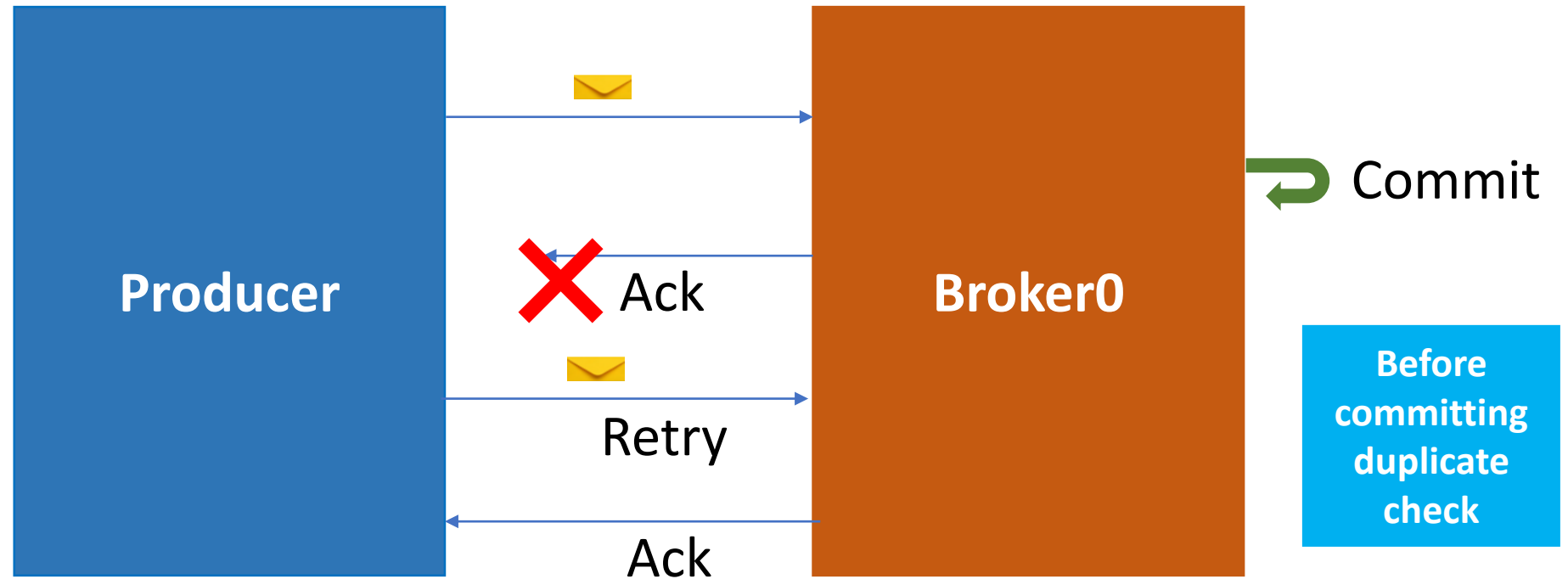
Idempotent Producer



Idempotent Producer

- We can avoid these duplicate messages by making Producer idempotent
enable.idempotence=true
- Idempotent Producer sends a unique message id with every message
- Before saving the message Broker uses that message id and performs duplicate check. If it is already committed – Broker willn't commit that message again and simply sends Acknowledgement to the Producer

Idempotent Producer



Producer Configurations

- **compression.type** – none/gzip/lz4/snappy
- **linger.ms**
 - It is the amount of time in milli seconds a Producer waits before sending the current batch.
 - The default value is 0 which means batches will be immediately sent even if there is only 1 message in the batch.
- **batch.size** – Maximum amount of memory in bytes that will be included in a batch

Producer Configurations

- **buffer.memory** – When the producer calls `send()`, the messages will not be immediately sent but added to an internal buffer. The default `buffer.memory` is 32MB. If the producer sends messages faster than they can be transmitted to the broker or there is a network issue, it will exceed `buffer.memory` and then the `send()` call will be blocked up to ***max.block.ms*** (default 1 minute).
- **max.block.ms** - This parameter controls how long the producer will block when `buffer.memory` is full. When `max.block.ms` is reached, a timeout exception is thrown.

Producer Configurations

- **max.in.flight.requests.per.connection** - This controls how many produce requests the producer can send to the server in parallel

Consumer Configurations

- **max.poll.records** – The maximum number of records returned in a single call to poll()
- **max.poll.interval** -Defines the maximum time between poll invocations. If it's not met, then the consumer considered as dead and rebalance happens

Consumer Configurations

- **session.timeout.ms**
 - Consumer periodically sends heartbeats to group coordinator
 - This is a mechanism to check if Consumer is alive / dead
 - If more than **session.timeout.ms** passes without the consumer sending a heartbeat to the group coordinator, it is considered dead, and rebalance is triggered.

Consumer Delivery Semantics

Consumer Delivery Semantics

- Atmost once
- Atleast once
- Exactly once

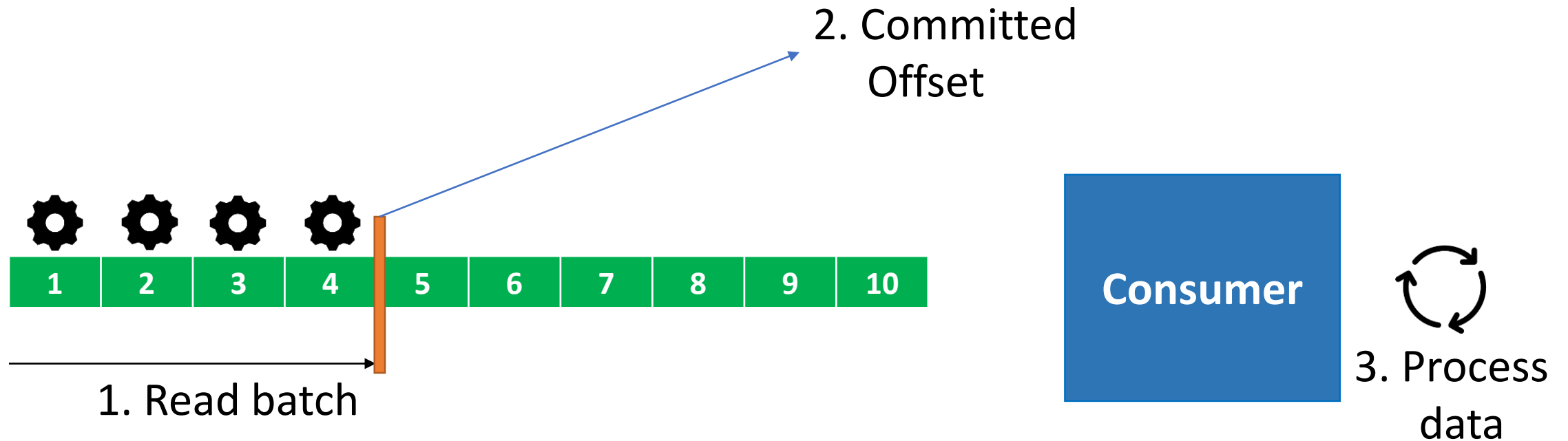
Consumer Delivery Semantics

At most once

- Offsets are committed as soon as message batch is received
- Problem with this approach is – If the processing goes wrong, then the message will be lost

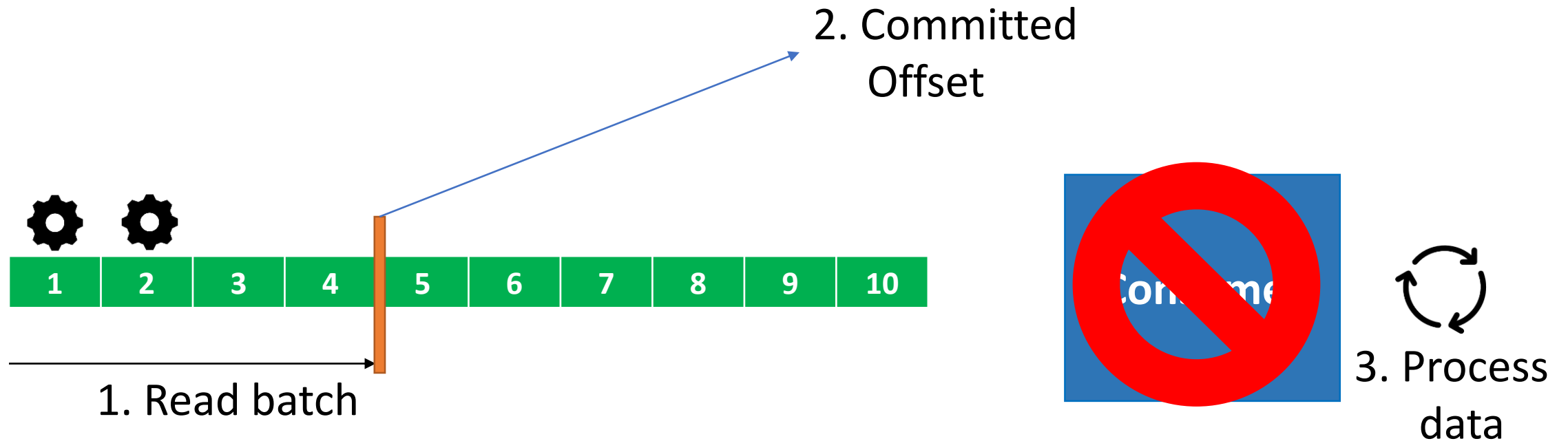
Consumer Delivery Semantics

At most once



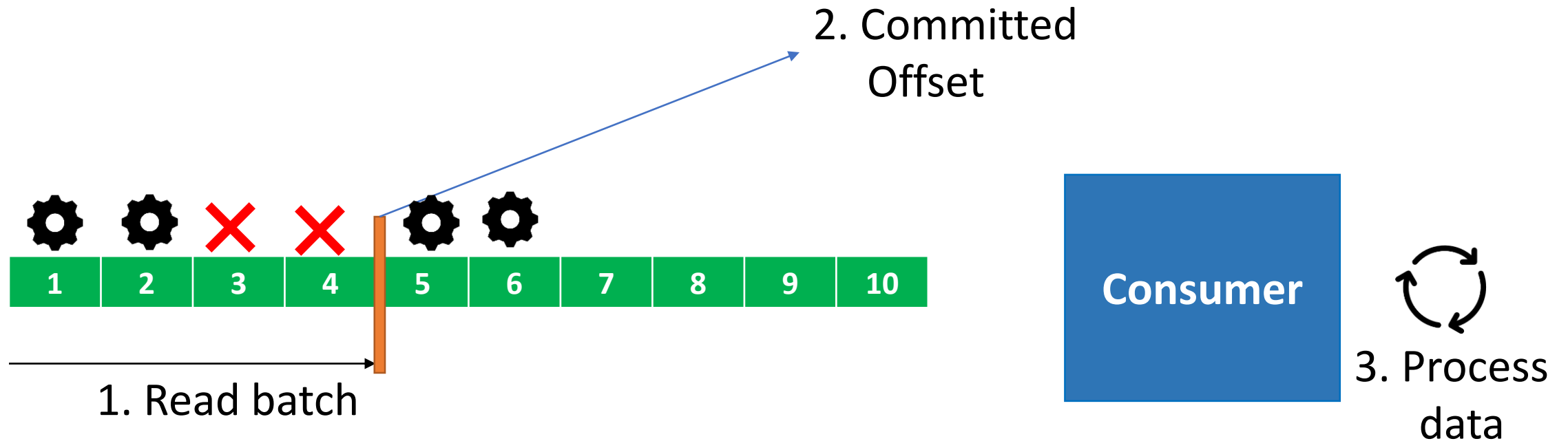
Consumer Delivery Semantics

At most once



Consumer Delivery Semantics

At most once



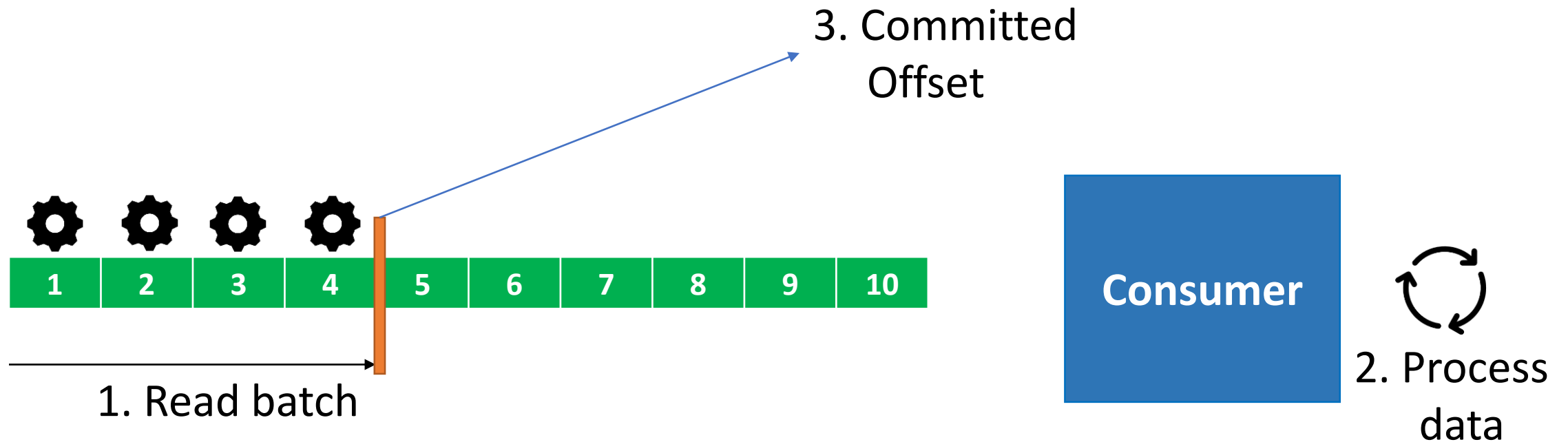
Consumer Delivery Semantics

At least once

- Offsets are committed after the message is processed
- Problem with this approach is – If the processing goes wrong, then the message will be read again which can result in duplicate processing of messages
- To overcome this problem, make consumer idempotent
- Idempotent means – processing the messages again willn't impact our system

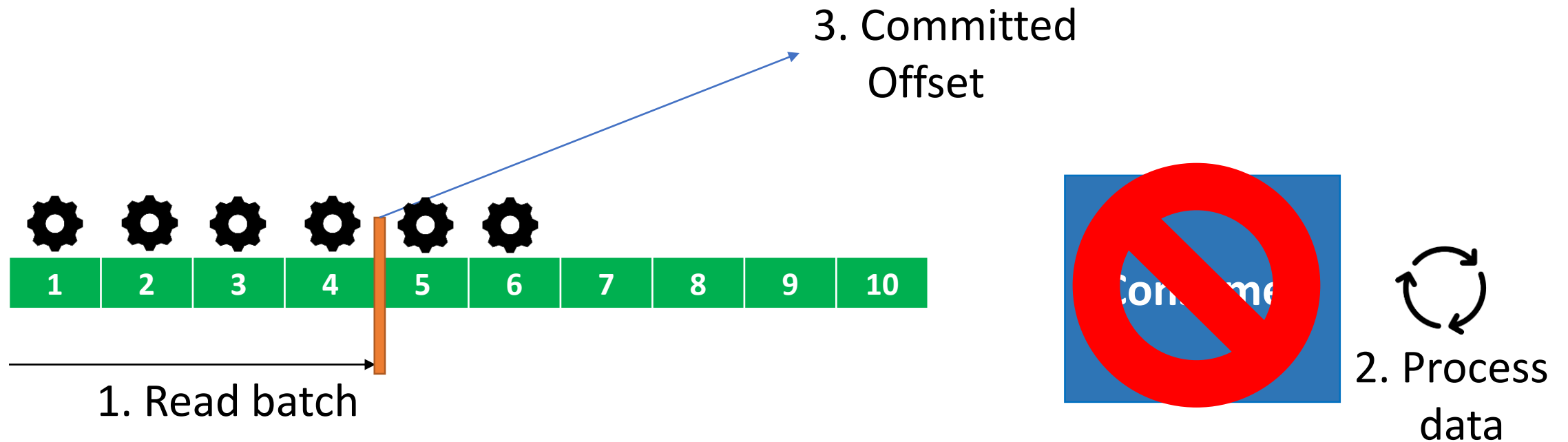
Consumer Delivery Semantics

At least once



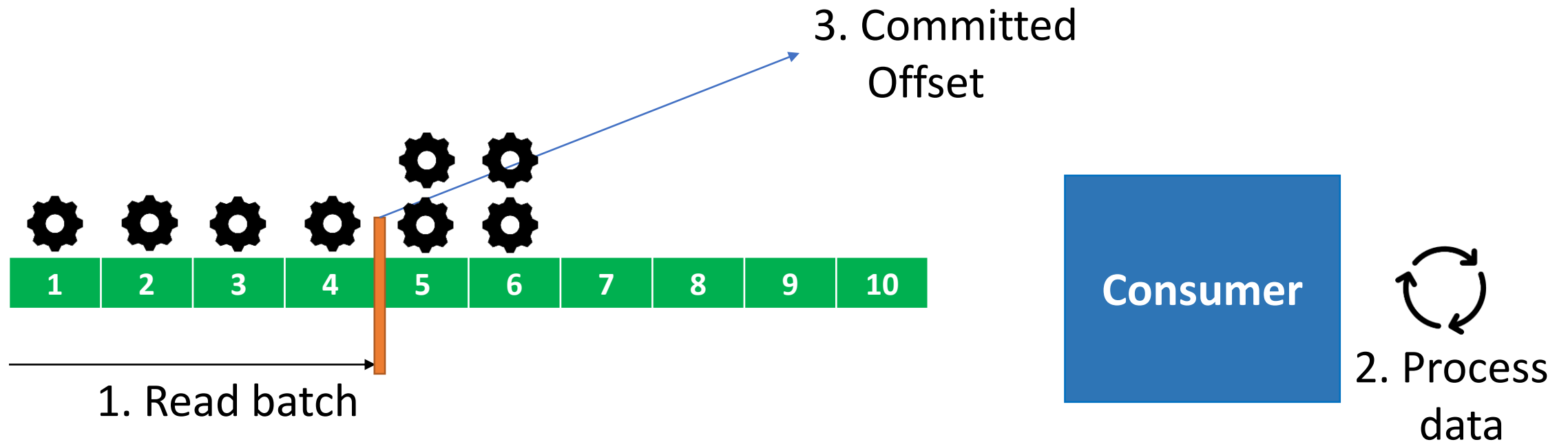
Consumer Delivery Semantics

At least once



Consumer Delivery Semantics

At least once



Consumer Delivery Semantics

Exactly once

- Can be achieved while using Kafka->Kafka workflows using Kafka Streams

Idempotent Consumer

- Messages getting processed twice can create issues
- **Ex.** Credit X amount to an account
- In this case, processing the same message more than once can create issues
- We can avoid this using Idempotent Consumer

Perform duplicate check

- Do a duplicate check before processing the message
- For example, create key as **topic+“-“+partition+“-“+offset** which is always unique
- During message processing save this key in database table say PROCESSED_MESSAGES
- Before performing any processing logic check if the Key is present in PROCESSED_MESSAGES. If so, it means it is a duplicate message. In that case don't process the message
- If it is not present, process the message

Perform duplicate check

```
while (true) {  
    Read message  
    Check if Key exists in PROCESSED_MESSAGE table  
    If exists  
        Then it is duplicate. Don't do any procesing logic  
    else  
        INSERT into PROCESSED_MESSAGE (ID) VALUES(messageID)  
        ... Continue processing logic...  
    end  
    Acknowledge message  
}
```

Offset Commit Strategies

- Auto commit (`enable.auto.commit=true`)
- Manual commit (`enable.auto.commit=false`)

Offset Commit Strategies

Auto commit (`enable.auto.commit=true`)

- By default, it is *true* which means that the consumer's offset will be periodically committed in the background
- Frequency of commit is determined by configuration `auto.commit.interval.ms`
- With auto commit enabled, a call to `poll()` will always commit the last offset returned by the previous poll

Offset Commit Strategies

Problem with Auto commit

- Data loss –
 - Say we enabled auto commit
 - Consumer received data and processing data asynchronously
 - Consumer thread calls poll() which performs offset commit also
 - What if asynchronous processing of previous batch fails?
 - As the offset already got committed, Consumer can't read that batch again

Offset Commit Strategies

Manual commit (`enable.auto.commit=false`)

- It provides a higher level of control to avoid data loss or data duplication
- Here consumer determines when a message is consumed
- We can do in 2 ways –
 - `commitSync()`
 - `commitAsync()`

Offset Commit Strategies – `commitSync()`

- It will commit the latest offset returned by `poll()`
- Returns once the offset is committed / throws an exception if commit fails
- Consumer calls `commitSync()` once it processed the current batch and waits for the commit confirmation before making another `poll()` call

Offset Commit Strategies – `commitSync()`

```
while (true) {  
    ConsumerRecords<String, String> records = consumer.poll(100);  
    for (ConsumerRecord<String, String> record : records)  
    {  
        // Processing logic  
    }  
    consumer.commitSync();  
}
```

Offset Commit Strategies – `commitSync()`

- One disadvantage with `commitSync()` is that the application is blocked until the broker responds to the commit request

Offset Commit Strategies – `commitAsync()`

- It will commit the latest offset returned by `poll()` asynchronously
- Instead of waiting for the broker to respond to a commit, we just send the request and continue on
- The `commitAsync` API has lower latency than the `commitSync` API, but risks creating duplicates when rebalancing (Because if commit fails – Consumer may read the same batch again)

Hands on

Confluent Kafka

AVRO & Schema Registry

Issues with out Contract enforcement Demo

How can we enforce contract?



AVRO

- We need to serialize data to transfer it over a network. To do this we need a serialization system
- Avro is a data serialization system
- Avro serializes the data into a binary format, which can be deserialized by any application
- Avro uses JSON format to declare the data structures

AVRO – Data Types

Supports both primitive & complex data types

- int
- long
- float
- double
- bytes
- string
- null

AVRO – Data Types

Complex data types include -

- Record
- Enum
- Arrays
- Unions
- Fixed

AVRO – Data Type - Record

Record is a collection of multiple attributes

Example –

```
{  
  "type" : "record",  
  "namespace" : "kafka.tutorial.demos",  
  "name" : "Employee",  
  "fields" : [  
    { "name" : "Name" , "type" : "string" },  
    { "name" : "age" , "type" : "int" }  
  ]  
}
```

AVRO – Data Type - Enum

An enumeration is a list of items in a collection

Example –

```
{  
  "type" : "enum",  
  "namespace" : "kafka.tutorial.demos",  
  "name" : "Day",  
  "symbols" : [ "SUNDAY", "MONDAY", "TUESDAY", "WEDNESDAY",  
                "THURSDAY", "FRIDAY", "SATURDAY" ]  
}
```

AVRO – Data Type - Array

An enumeration is a list of items in a collection

Example –

```
{  
  "type" : " array ",  
  "items" : " int "  
}
```


AVRO – Data Type - Map

The map data type is an array of key-value pairs

Key of an AVRO map must be a string

Example –

```
{  
  "type" : " map",  
  "values" : " int "  
}
```

AVRO – Data Type - Union

A union datatype is used whenever the field has one or more datatypes
They are represented as JSON arrays.

For example, if a field that could be either an int or null, then the union is represented as ["int", "null"].

AVRO – Data Type - Union

```
{  
  "type" : "record",  
  "namespace" : "kafka.tutorial.demos",  
  "name" : "FullName",  
  "fields" : [  
    { "name" : "firstname", "type" : "string" },  
    { "name" : "middlename", "type" : ["string", "null"] },  
    { "name" : "lastname", "type": "string" }  
  ]  
}
```

AVRO – Data Type - Fixed

A fixed type is used to declare a fixed-sized field

Example –

```
{  
  "type": "fixed",  
  "size": 16,  
  "name": "md5"  
}
```

AVRO – Data Type - Fixed

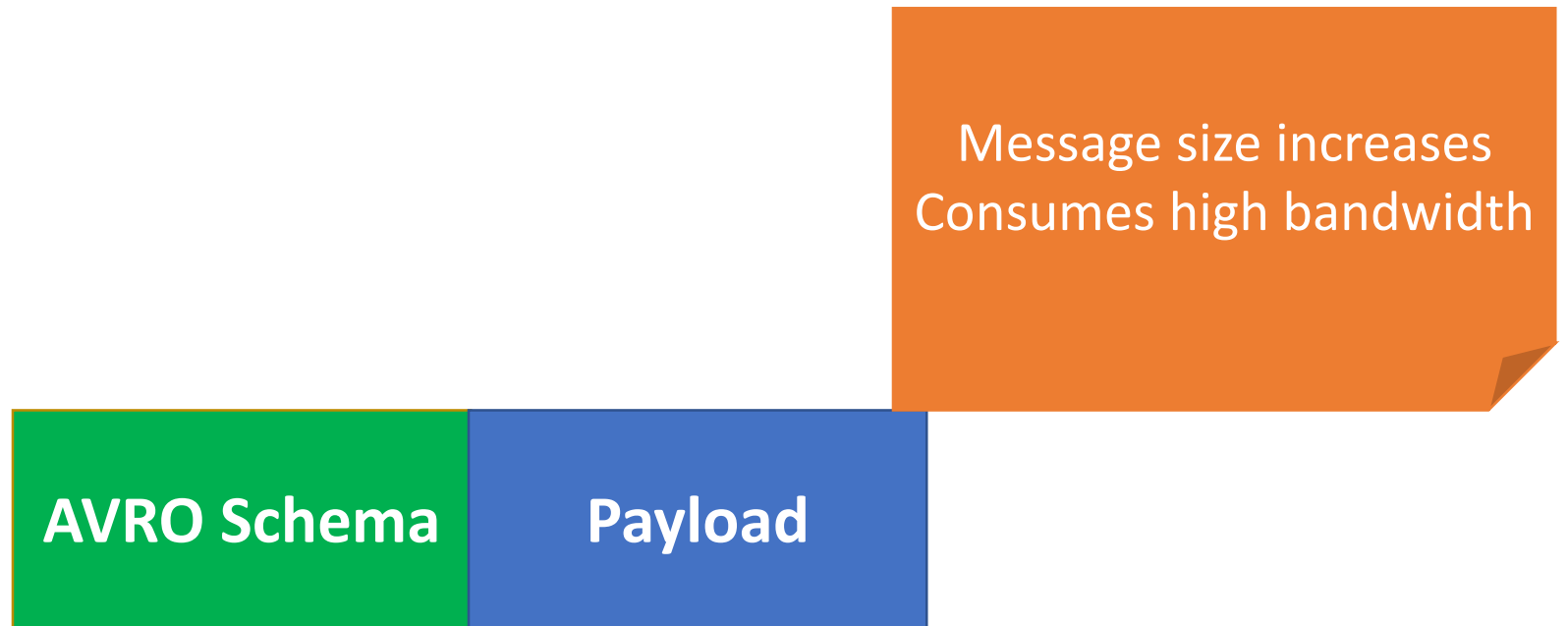
A fixed type is used to declare a fixed-sized field

Example –

```
{  
  "type": "fixed",  
  "size": 16,  
  "name": "md5"  
}
```

AVRO

AVRO requires each message to contain schema payload which will be used during deserialization



AVRO

- As it is a serialization system it performs serialization and deserialization
- So along with AVRO we need something else to
 - enforce contracts
 - support schema evolution
 - reducing the payload size
 - easy maintenance of schemas

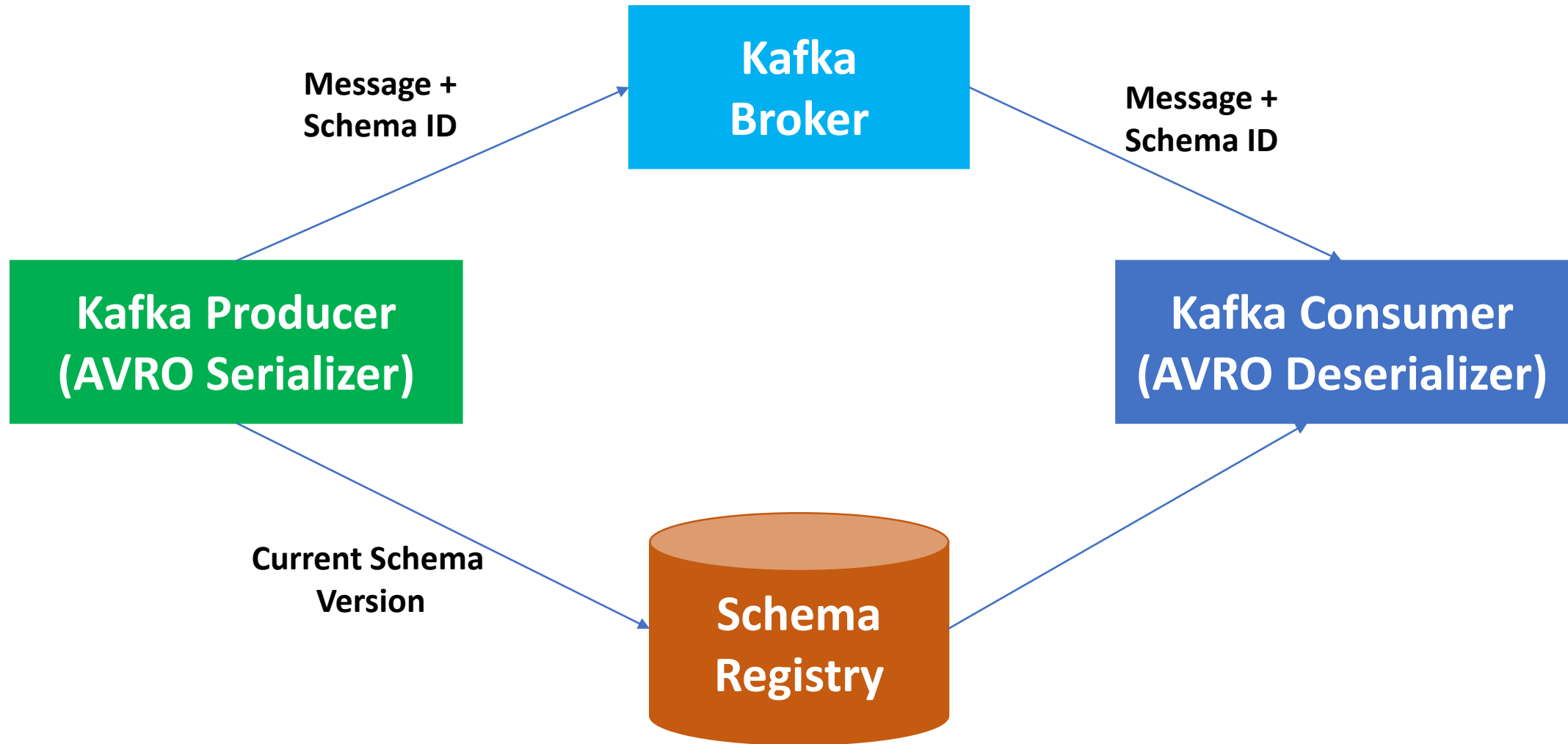
Use *Schema Registry Pattern*



Schema Registry

- The idea is to store all the schemas used to write data to Kafka in the registry
- Store the identifier for the schema in the Kafka message and send
- Consumer then use the identifier to pull the record out of the schema registry and deserialize the data
- This will be done by AVRO Serializer, Deserializer automatically

Schema Registry



Schema Registry - Uses

- Enforcing Data Contracts
- Schema Evolution
- Compatibility Checks

Schema Registry – Compatibility Modes

- BACKWARD
- BACKWARD_TRANSITIVE
- FORWARD
- FORWARD_TRANSITIVE
- FULL
- FULL_TRANSITIVE
- NONE

Schema Registry – Compatibility Modes

Compatibility Type	Changes allowed	Check against which schemas	Upgrade first
BACKWARD	• Delete fields • Add optional fields	Last version	Consumers
BACKWARD_TRANSITIVE	• Delete fields • Add optional fields	All previous versions	Consumers

Schema Registry – Compatibility Modes

Compatibility Type	Changes allowed	Check against which schemas	Upgrade first
FORWARD	• Add fields • Delete optional fields	Last version	Producers
FORWARD_TRANSITIVE	• Add fields • Delete optional fields	All previous versions	Producers

Schema Registry – Compatibility Modes

Compatibility Type	Changes allowed	Check against which schemas	Upgrade first
FULL	• Add optional fields • Delete optional fields	Last version	Any order
FULL_TRANSITIVE	• Add optional fields • Delete optional fields	All previous versions	Any order

Schema Registry – Compatibility Modes

Compatibility Type	Changes allowed	Check against which schemas	Upgrade first
NONE	• All changes accepted	No check	Depends (Mostly both at same time)

Schema Registry – REST END Points

- To get global compatibility mode

Method – GET

URL - **<http://localhost:8081/config/>**

- To update global compatibility mode

Method – PUT

URL - **<http://localhost:8081/config/>**

Schema Registry – REST END Points

- To get compatibility mode of a schema

Method – GET

URL - **`http://localhost:8081/config/subjectname/`**

- To update global compatibility mode

Method – PUT

URL - **`http://localhost:8081/config/ subjectname/`**

Reference

<https://docs.confluent.io/platform/current/schema-registry/develop/api.html>

Schema Registry

In my understanding, schema registry means - it is not a replacement of our jar files.

Earlier my understanding was - avro schema gets stored in this registry and .class files get generated on the fly in Producer and Consumer end based

on schema. But this seems NOT correct.

Subject is parent entity

Schema is child entity

We do have centralized schemas repo which generates .java, .class files and creates artefact.

This artefact will be used in Producer, Consumer pom.xml files

If Producer want to make any change then he changes avro avsc files and regenerate artefact and publish.

At same time, schema version will be incremented in schema-registry.

For this I think we have 2 strategies. One is as and when we change the avro and regenerate it pushed into schema-registry.

Otherwise - we do first make changes in schema repo and push the same changes to schema-registry using schema rest end points.

Before doing this change, I think we can check schema compatibility also as per the given compatibility mode.

This is a REST End Point of schema-registry.

Now what is the use of schema-registry then?

If it is BACKWARD compatible - Then it means we can add Optional field / Delete required field - Consumer need to be upgraded first

If it is FORWARD compatible - Then it means we can delete optional field / Add required field - Producer need to be upgraded first

If it is FULL compatible - Highly restrictive. it means only optional field can be added / optional field can be deleted -upgrade can happen any order

If it is NONE compatible - Then we can do whatever we want. Add/Delete optional fields, Add/Delete required fields. But both Producer, Consumer should be upgraded first

With schema-registry we are getting this flexibility

Not only this, if Producer tries to publish some message which is not compatible it fails (This one we can see if we put auto schema registry option to false I think.

Else in my opinion it simply try to create a new schema version with these changes)

One more benefit is - Contract Enforcement in general.

Same Topic can get data from multiple Producers. Now at Consumer end we can simply use GenericRecord and using instanceOf we can process accordingly.

Say Vendor Items Topic from multiple Vendors and Legacy system.

Now question is how can we say that we are enforcing contracts?

Because the moment Producer tries to add some mandatory field or change some datatype - schema registry checks the COMPATIBILITY mode and

it fails. Now to make it work - we need to change the COMPATIBILITY mode to NONE both Producer & Consumer should upgrade at same time.

One more benefit is - Producer need not send entire schema definition in message for the AvroDeserializer to use at Consumer end.

It simply sends schema id. And using that schema id AvroDeserializer checks cache and get the schema definition and deserializes.

If it can't find then it goes to schema registry to get the schema for that particular version, uses it and caches it

As an architecture pattern, a schema registry is quite simple and has only two components:

REST service for storing and retrieving schemas

Java library for fetching and caching schemas

When a producer produces an Avro event, we take the schema of the event and look for it in the schema registry. If the schema is new, we register it in the schema registry and generate an ID. If the schema already exists, we take its ID from the registry. Either way, every time we produce an event with that schema, we store the ID together with the event. Note that the schema and the ID are cached, so as long as we keep producing events with the same schema, we don't need to talk to the schema registry again.

When a consumer encounters an event with a schema ID, it looks the schema up in the schema registry using the ID, then caches the schema, and uses it to deserialize every event it consumes with this ID.

Demo about subject name strategy/ record name strategy

<https://www.cloudera.com/tutorials/schema-registry-in-trucking-iot/1.html#>

<https://chrisrichardson.net/post/microservices/patterns/2020/10/16/idempotent-consumer.html>

One more approach is to maintain list of unique keys inside mongodb document for example -

Idempotent Consumer Another Approach

<https://chrisrichardson.net/post/microservices/patterns/2020/10/16/idempotent-consumer.html>

One more approach is to maintain list of unique keys inside mongodb document for example -

```
{  
  AccountID:  
  Balance:  
  MessageIDs:  
  {  
    ACCOUNT_CREDIT_EVENT_PARTITION1_1,  
    ACCOUNT_CREDIT_EVENT_PARTITION1_2,  
    ACCOUNT_CREDIT_EVENT_PARTITION1_3  
  }  
}
```

Now while inserting or updating our Consumer process only if the document MessageIDs doesn't contain the MessageID.